MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Módulo 7 - Tarea 2

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- Entendiendo que pide la tarea
- Conexión con la MV
- Identificar qué es crypy
- Análisis de los strings de crypy
- Desensamblando crypy
- Dónde estuvo la clave
- Implicación para el análisis de memoria
- Buscar material PEM y Nombres sospechosos en disco
- Comprobar si crypy sigue vivo o dejó trazas simples
- Revisar directorios temporales
- Revisar el entorno Python y las utilidades RSA instaladas
- Escaneo de la memoria física activa
- Escaneo de la memoria física
- Localización por desplazamiento - Offset
- Localización por proc-kcore
- Escaneo profundo de la partición de sistema
- Lista de procesos
- Buscamos los rastros que deja crypy
- La pista de la Clave Fragmentada
- Encontrando diversas claves PEM en la memoria
- Cambio de Estrategia: Recuperar crypy
    - Volcado de Memoria RAM con AVML
    - Copia de la swap
    - Extracción de cadenas imprimibles
    - Búsqueda de artefactos relevantes
    - Buscamos si keys.txt existe
    - Recuperamos el marcador Keys.txt de la Swap
    - Sacamos esa ventana exacta de swap
    - Buscamos restos del .pyc
    - Recordando el fichero crypy
    - Desensamblamos el fichero crypy
    - Buscamos cabeceras, pies y Base64 dentro de swap_keys_region.txt
    - Obtenemos una clave candidata
    - Reconstruimos la región de swap correspondiente al contenido de keys.txt
    - La clave private NO está completa
    - LA PEM RECONSTRUIDA
- La solución
- Conclusiones

Entendiendo que pide la tarea

La tarea consiste en importar la máquina virtual, acceder a ella por SSH, buscar en memoria la clave privada RSA generada por el proceso `crypy`, utilizar esa clave en `crypy_decryptor.py` para descifrar `mysecrets.txt.encrypted` y documentar todo el procedimiento en un informe.

Pasos aproximados:

- Importar la máquina OVA en VirtualBox.
- Configurar la red en modo Bridge para que la VM obtenga una IP visible desde tu máquina host.
- Arrancar la VM e iniciar sesión localmente para comprobar la IP con ifconfig, fijándonos en la interfaz `enp0s17`.
- Conectarse por SSH desde nuestro equipo host a esa IP. El objetivo indica usar admin1/1234, aunque en las instrucciones de partida aparece admin2/1234; si uno no funciona.
- Realizar pruebas de análisis de memoria para localizar restos del proceso crypy y recuperar la clave privada RSA. El propio enunciado indica que la memoria puede dar fragmentos parciales, pero útiles para reconstruir la clave.
- Buscar la clave con formato PEM, identificando el bloque que empieza por -----BEGIN RSA PRIVATE KEY----- y termina por -----END RSA PRIVATE KEY-----.
- Copiar la clave privada dentro del script `crypy_decryptor.py`, rellenando la constante `PRIV_KEY`.
- Ejecutar el script para descifrar mysecrets.txt.encrypted y generar mysecrets.txt.
- Verificar el contenido de mysecrets.txt, que debe ser un mensaje con sentido.

El informe debe contar con:

- Todas las pruebas realizadas para buscar la huella en memoria,
- dónde y cómo localizaste/extrajiste la clave,
- la clave privada obtenida,
- el contenido final del fichero descifrado,
- y capturas de pantalla que documenten el proceso.

Conexión con la MV

mru8-ej2 [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

```
 * Support:        https://ubuntu.com/advantage

 System information disabled due to load higher than 1.0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

278 packages can be updated.
203 updates are security updates.


[  OK  ] Created slice User Slice of admin1.
         Starting User Manager for UID 1000...
[  OK  ] Started Session 1 of user admin1.
[  OK  ] Started User Manager for UID 1000.
admin1@mru8:~$ ifconfig
enp0s17: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.144.254  netmask 255.255.255.0  broadcast 192.168.144.255
        inet6 fe80::a00:27ff:fe6e:ed99  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:6e:ed:99  txqueuelen 1000  (Ethernet)
        RX packets 69470  bytes 104735051 (104.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2967  bytes 235512 (235.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 122  bytes 10850 (10.8 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 122  bytes 10850 (10.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

admin1@mru8:~$ _
```

```
admin1@mru8: ~

┌──(xxniwexx㉿xxniwexx)-[~/Escritorio/cybersecurity]
└─$ ssh admin1@192.168.144.254
The authenticity of host '192.168.144.254 (192.168.144.254)' can't be established.
ED25519 key fingerprint is: SHA256:gH5sGozwBLV3c+z9N+PxwPBh65ZieJwnJUnhwQgVH54
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.144.254' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
admin1@192.168.144.254's password:
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-76-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 System information disabled due to load higher than 1.0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

278 packages can be updated.
203 updates are security updates.

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Fri Mar 20 16:03:30 2026
admin1@mru8:~$
```

Identificar qué es crypy

```
admin1@mru8:~$ file /home/admin1/crypy
/home/admin1/crypy: python 3.6 byte-compiled
admin1@mru8:~$ stat /home/admin1/crypy
  File: /home/admin1/crypy
  Size: 2516        Blocks: 8          IO Block: 4096   regular file
Device: 802h/2050d  Inode: 792780      Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  admin1)   Gid: ( 1000/  admin1)
Access: 2026-03-20 15:11:07.236268689 +0000
Modify: 2021-01-19 01:13:22.712365137 +0000
Change: 2021-01-19 01:15:59.336219431 +0000
 Birth: -
admin1@mru8:~$ sha256sum /home/admin1/crypy
3565edeaf24f3595a6a53342ee7e6bf12a5c0fbbf50b1f61cbeefc9809500bf3
/home/admin1/crypy
```

donde:

- Confirmamos que `crypy` no es un binario nativo (ELF), sino bytecode de Python 3.6, es decir, el equivalente a un `.pyc`.

Análisis de los strings de crypy

```
root@mru8:~# strings /home/admin1/crypy
m   Z   m
RSA)
PKCS1_OAEP)
PKCS1_v1_5)
SHA512
SHA384
SHA256
MD5)
Random)
    b64encode
    b64decode
SHA-256c
readr
generate
    publickey)
keysizeZ
random_generator
private
public
```

```
crypy.py
newkeys
    importKey)
Z   externKeyr
priv_keyr
getpublickey$
encrypt)
message
pub_key
cipherr
decrypt)
ciphertextr
SHA-512z
SHA-384z
SHA-256z
SHA-1)
hashr
update
sign)
hashAlg
signer
digestr
SHA-512z
SHA-384z
SHA-256z
SHA-1)
verify)
Z   signaturer
)   Nz
./mysecrets.txt
keys.txt
./mysecrets.txt.encrypted
openr
rsar
save_pkcs1
write
decoder
remove) Z
msg1r
public_pemZ
private_pem
fZ  encryptedZ
encrypted_filer
mainQ
__main__i?B
Crypto.PublicKeyr
Crypto.Cipherr
Crypto.Signaturer
Crypto.Hashr
Cryptor
base64r
timer
__name__Z
```

```
sleepr
<module>
```

donde:

- Se confirma que `crypy` trabajaba con material criptográfico RSA. Entre las cadenas más relevantes aparecen:
  - `RSA`
  - `publickey`
  - `newkeys`
  - `generate`
  - `private`
  - `public`
  - `importKey`
  - `save_pkcs1`
  - `public_pem`
  - `private_pem`
- Estas cadenas sugieren que el programa no sólo utilizaba cifrado, sino que además generaba un par de claves RSA, lo serializaba en formato `PKCS#1/PEM` y lo manejaba dentro del propio script.
- Parece que llegó a escribir las claves en disco: Estas cadenas son especialmente relevantes:
  - `keys.txt`
  - `save_pkcs1`
  - `write`
  - `remove`
- Flujo bastante plausible:
  - generar claves;
  - exportarlas en `PKCS#1/PEM`;
  - escribirlas en un fichero llamado `keys.txt`;
  - y borrar ese fichero después con `remove`.
- Objetivo del cifrado: Estas cadenas encajan perfectamente con el enunciado:
  - `./mysecrets.txt`.
  - `./mysecrets.txt.encrypted`
  - `encrypt`
  - `decrypt`
  - `cipher`
  - `ciphertext`
- Eso apunta a que `crypy`:
  - tomaba `mysecrets.txt,`,
  - generaba el contenido cifrado,
  - y escribía `mysecrets.txt.encrypted`.

Conclusión: El análisis de cadenas del archivo `crypy` mostró indicios sólidos de que el programa implementaba generación de claves RSA y su serialización en formato `PKCS#1/PEM`. Entre las cadenas más relevantes destacan `newkeys`, `generate`, `private_pem`, `public_pem` y `save_pkcs1`. Además, la presencia conjunta de `keys.txt`, `write` y `remove` sugiere que las claves pudieron escribirse temporalmente en disco y ser eliminadas posteriormente. También se identificaron referencias directas a `mysecrets.txt` y `mysecrets.txt.encrypted`, coherentes con la operativa de cifrado descrito en el enunciado del ejercicio.

Desensamblando crypy

Script en python para desensamblar el bytecode para ver el flujo lógico de crypy:

```
import marshal, dis

path = "/home/admin1/crypy"
with open(path, "rb") as f:
    f.read(12)
    code = marshal.load(f)

dis.dis(code)
```

Resultado:

```
admin1@mru8:~$ python3 desensamblado.py
 14           0 LOAD_CONST               0 (0)
              2 LOAD_CONST               1 (None)
              4 IMPORT_NAME              0 (os)
              6 STORE_NAME               0 (os)

 16           8 LOAD_CONST               0 (0)
             10 LOAD_CONST               2 (('RSA',))
             12 IMPORT_NAME              1 (Crypto.PublicKey)
             14 IMPORT_FROM              2 (RSA)
             16 STORE_NAME               2 (RSA)
             18 POP_TOP

 17          20 LOAD_CONST               0 (0)
             22 LOAD_CONST               3 (('PKCS1_OAEP',))
             24 IMPORT_NAME              3 (Crypto.Cipher)
             26 IMPORT_FROM              4 (PKCS1_OAEP)
             28 STORE_NAME               4 (PKCS1_OAEP)
             30 POP_TOP

 18          32 LOAD_CONST               0 (0)
             34 LOAD_CONST               4 (('PKCS1_v1_5',))
```

```
                36 IMPORT_NAME              5 (Crypto.Signature)
                38 IMPORT_FROM              6 (PKCS1_v1_5)
                40 STORE_NAME               6 (PKCS1_v1_5)
                42 POP_TOP

  19            44 LOAD_CONST               0 (0)
                46 LOAD_CONST               5 (('SHA512', 'SHA384', 'SHA256',
'SHA', 'MD5'))
                48 IMPORT_NAME              7 (Crypto.Hash)
                50 IMPORT_FROM              8 (SHA512)
                52 STORE_NAME               8 (SHA512)
                54 IMPORT_FROM              9 (SHA384)
                56 STORE_NAME               9 (SHA384)
                58 IMPORT_FROM             10 (SHA256)
                60 STORE_NAME              10 (SHA256)
                62 IMPORT_FROM             11 (SHA)
                64 STORE_NAME              11 (SHA)
                66 IMPORT_FROM             12 (MD5)
                68 STORE_NAME              12 (MD5)
                70 POP_TOP

  20            72 LOAD_CONST               0 (0)
                74 LOAD_CONST               6 (('Random',))
                76 IMPORT_NAME             13 (Crypto)
                78 IMPORT_FROM             14 (Random)
                80 STORE_NAME              14 (Random)
                82 POP_TOP

  21            84 LOAD_CONST               0 (0)
                86 LOAD_CONST               7 (('b64encode', 'b64decode'))
                88 IMPORT_NAME             15 (base64)
                90 IMPORT_FROM             16 (b64encode)
                92 STORE_NAME              16 (b64encode)
                94 IMPORT_FROM             17 (b64decode)
                96 STORE_NAME              17 (b64decode)
                98 POP_TOP

  22           100 LOAD_CONST               0 (0)
               102 LOAD_CONST               1 (None)
               104 IMPORT_NAME             18 (rsa)
               106 STORE_NAME              18 (rsa)

  23           108 LOAD_CONST               0 (0)
               110 LOAD_CONST               1 (None)
               112 IMPORT_NAME             19 (time)
               114 STORE_NAME              19 (time)

  25           116 LOAD_CONST               8 ('SHA-256')
               118 STORE_GLOBAL            20 (hash)

  27           120 LOAD_CONST               9 (<code object newkeys at
0x7f6dba57ba50, file "crypy.py", line 27>)
               122 LOAD_CONST              10 ('newkeys')
               124 MAKE_FUNCTION            0
```

```
              126 STORE_NAME              21 (newkeys)

 33           128 LOAD_CONST              11 (<code object importKey at
0x7f6dba5125d0, file "crypy.py", line 33>)
              130 LOAD_CONST              12 ('importKey')
              132 MAKE_FUNCTION            0
              134 STORE_NAME              22 (importKey)

 36           136 LOAD_CONST              13 (<code object getpublickey at
0x7f6dba5124b0, file "crypy.py", line 36>)
              138 LOAD_CONST              14 ('getpublickey')
              140 MAKE_FUNCTION            0
              142 STORE_NAME              23 (getpublickey)

 39           144 LOAD_CONST              15 (<code object encrypt at
0x7f6dba512a50, file "crypy.py", line 39>)
              146 LOAD_CONST              16 ('encrypt')
              148 MAKE_FUNCTION            0
              150 STORE_NAME              24 (encrypt)

 44           152 LOAD_CONST              17 (<code object decrypt at
0x7f6dba5129c0, file "crypy.py", line 44>)
              154 LOAD_CONST              18 ('decrypt')
              156 MAKE_FUNCTION            0
              158 STORE_NAME              25 (decrypt)

 49           160 LOAD_CONST              27 (('SHA-256',))
              162 LOAD_CONST              19 (<code object sign at
0x7f6dba512ae0, file "crypy.py", line 49>)
              164 LOAD_CONST              20 ('sign')
              166 MAKE_FUNCTION            1
              168 STORE_NAME              26 (sign)

 66           170 LOAD_CONST              21 (<code object verify at
0x7f6dba512b70, file "crypy.py", line 66>)
              172 LOAD_CONST              22 ('verify')
              174 MAKE_FUNCTION            0
              176 STORE_NAME              27 (verify)

 81           178 LOAD_CONST              23 (<code object main at
0x7f6dba512c00, file "crypy.py", line 81>)
              180 LOAD_CONST              24 ('main')
              182 MAKE_FUNCTION            0
              184 STORE_NAME              28 (main)

119           186 LOAD_NAME               29 (__name__)
              188 LOAD_CONST              25 ('__main__')
              190 COMPARE_OP               2 (==)
              192 POP_JUMP_IF_FALSE      210

120           194 LOAD_NAME               28 (main)
              196 CALL_FUNCTION            0
              198 POP_TOP
```

```
121          200 LOAD_NAME                19 (time)
             202 LOAD_ATTR                30 (sleep)
             204 LOAD_CONST               26 (999999)
             206 CALL_FUNCTION             1
             208 POP_TOP
         >>  210 LOAD_CONST                1 (None)
             212 RETURN_VALUE
```

A destacar:

- `__name__ == "__main__"`
- `main()`
- `time.sleep(999999)`
- Donde vemos que `crypy` no termina al instante después de ejecutar `main()`. Se queda durmiendo muchísimo tiempo. Eso sugiere que, al lanzarse como programa principal, hacía su trabajo en `main()` y luego permanece vivo en memoria. El proceso pudiera dejar material criptográfico recuperable en RAM.

El bytecode muestra que crypy ejecuta la función main() y, a continuación, entra en una suspensión prolongada (sleep(999999)). Este comportamiento resulta coherente con un proceso que, tras realizar las operaciones criptográficas principales, permanece residente en memoria, lo que refuerza la hipótesis de que pudo dejar en RAM material criptográfico reutilizable o recuperable

---

Dónde estuvo la clave

Vamos a reconstruir la lógica de crypy para demostrar dónde estuvo la clave, cómo se exportó y qué rastro dejó. Vamos a reconstruir la lógica de crypy para demostrar dónde estuvo la clave, cómo se exportó y qué rastro dejó.

Script para obtener el desensamblado de las funciones de crypy:

```
import marshal, dis, types

path = "/home/admin1/crypy"
with open(path, "rb") as f:
    f.read(12)
    code = marshal.load(f)

def walk(codeobj):
    print(f"\n=== CODE OBJECT: {codeobj.co_name} ===")
    dis.dis(codeobj)
    for const in codeobj.co_consts:
        if isinstance(const, types.CodeType):
            walk(const)

walk(code)
```

Salida:

```
root@mru8:~# python3 busca_funciones.py

=== CODE OBJECT: <module> ===
 14           0 LOAD_CONST               0 (0)
              2 LOAD_CONST               1 (None)
              4 IMPORT_NAME              0 (os)
              6 STORE_NAME               0 (os)

 16           8 LOAD_CONST               0 (0)
             10 LOAD_CONST               2 (('RSA',))
             12 IMPORT_NAME              1 (Crypto.PublicKey)
             14 IMPORT_FROM              2 (RSA)
             16 STORE_NAME               2 (RSA)
             18 POP_TOP

 17          20 LOAD_CONST               0 (0)
             22 LOAD_CONST               3 (('PKCS1_OAEP',))
             24 IMPORT_NAME              3 (Crypto.Cipher)
             26 IMPORT_FROM              4 (PKCS1_OAEP)
             28 STORE_NAME               4 (PKCS1_OAEP)
             30 POP_TOP
```

```
...
...
=== CODE OBJECT: main ===
 82           0 LOAD_GLOBAL              0 (open)
              2 LOAD_CONST               1 ('./mysecrets.txt')
              4 LOAD_CONST               2 ('rb')
              6 CALL_FUNCTION            2
              8 LOAD_ATTR                1 (read)
             10 CALL_FUNCTION            0
             12 STORE_FAST               0 (msg1)

 84          14 LOAD_CONST               3 (2048)
             16 STORE_FAST               1 (keysize)

 86          18 LOAD_GLOBAL              2 (rsa)
             20 LOAD_ATTR                3 (newkeys)
             22 LOAD_FAST                1 (keysize)
             24 CALL_FUNCTION            1
             26 UNPACK_SEQUENCE          2
             28 STORE_FAST               2 (public)
             30 STORE_FAST               3 (private)

 88          32 LOAD_FAST                2 (public)
             34 LOAD_ATTR                4 (save_pkcs1)
             36 CALL_FUNCTION            0
             38 STORE_FAST               4 (public_pem)

 89          40 LOAD_FAST                3 (private)
             42 LOAD_ATTR                4 (save_pkcs1)
             44 CALL_FUNCTION            0
             46 STORE_FAST               5 (private_pem)

 91          48 LOAD_GLOBAL              0 (open)
             50 LOAD_CONST               4 ('keys.txt')
             52 LOAD_CONST               5 ('w')
             54 CALL_FUNCTION            2
             56 SETUP_WITH              44 (to 102)
             58 STORE_FAST               6 (f)

 92          60 LOAD_FAST                6 (f)
             62 LOAD_ATTR                5 (write)
             64 LOAD_FAST                4 (public_pem)
             66 LOAD_ATTR                6 (decode)
             68 CALL_FUNCTION            0
             70 CALL_FUNCTION            1
             72 POP_TOP

 93          74 LOAD_FAST                6 (f)
             76 LOAD_ATTR                5 (write)
             78 LOAD_CONST               6 ('\n')
             80 CALL_FUNCTION            1
             82 POP_TOP

 94          84 LOAD_FAST                6 (f)
```

```
                 86 LOAD_ATTR                5 (write)
                 88 LOAD_FAST                5 (private_pem)
                 90 LOAD_ATTR                6 (decode)
                 92 CALL_FUNCTION            0
                 94 CALL_FUNCTION            1
                 96 POP_TOP
                 98 POP_BLOCK
                100 LOAD_CONST               0 (None)
           >>   102 WITH_CLEANUP_START
                104 WITH_CLEANUP_FINISH
                106 END_FINALLY

  99            108 LOAD_GLOBAL              7 (b64encode)
                110 LOAD_GLOBAL              2 (rsa)
                112 LOAD_ATTR                8 (encrypt)
                114 LOAD_FAST                0 (msg1)
                116 LOAD_FAST                3 (private)
                118 CALL_FUNCTION            2
                120 CALL_FUNCTION            1
                122 STORE_FAST               7 (encrypted)

 114            124 LOAD_GLOBAL              0 (open)
                126 LOAD_CONST               7 ('./mysecrets.txt.encrypted')
                128 LOAD_CONST               8 ('wb')
                130 CALL_FUNCTION            2
                132 SETUP_WITH              16 (to 150)
                134 STORE_FAST               8 (encrypted_file)

 115            136 LOAD_FAST                8 (encrypted_file)
                138 LOAD_ATTR                5 (write)
                140 LOAD_FAST                7 (encrypted)
                142 CALL_FUNCTION            1
                144 POP_TOP
                146 POP_BLOCK
                148 LOAD_CONST               0 (None)
           >>   150 WITH_CLEANUP_START
                152 WITH_CLEANUP_FINISH
                154 END_FINALLY

 117            156 LOAD_GLOBAL              9 (os)
                158 LOAD_ATTR               10 (remove)
                160 LOAD_CONST               1 ('./mysecrets.txt')
                162 CALL_FUNCTION            1
                164 POP_TOP
                166 LOAD_CONST               0 (None)
                168 RETURN_VALUE
```

donde:

- Lee el fichero original: `open('./mysecrets.txt', 'rb').read()`.
- Guarda el contenido en msg1.

- Fija tamaño de clave: `keysize = 2048`.
- Genera un par RSA: `public, private = rsa.newkeys(2048)`.
- Exporta ambas claves a PEM/PKCS#1:
  - `public_pem = public.save_pkcs1()`.
  - `private_pem = private.save_pkcs1()`.
- Escribe las dos claves en `keys.txt`.
- Cifra el contenido: `encrypted = b64encode(rsa.encrypt(msg1, private))`.
  - Pasa: `private` a `rsa.encrypt()`.
- Guarda el resultado: `open('./mysecrets.txt.encrypted', 'wb').write(encrypted)`.
- Borra el original: `os.remove('./mysecrets.txt')`.
- El proceso duerme muchísimo, fuera de `main()`: `time.sleep(999999)`.

Conclusiones importantes:

- La clave privada existió claramente en tres formas:
  - Como objeto RSA en memoria: `private`.
  - Como PEM serializada en memoria: `private_pem = private.save_pkcs1()`.
  - Como texto escrito en `keys.txt`: `f.write(private_pem.decode())`.

Implicación para el análisis de memoria

Aunque main() termine, el módulo hace luego sleep(999999). Eso implica que el proceso permanecía vivo durante mucho tiempo.

Aunque las variables locales de `main()` salgan de alcance al terminar la función:

- Los bytes de `private_pem` pueden quedar residualmente en el heap del proceso de crypy mientras estaba ejecutándose, es decir, a la memoria dinámica del intérprete Python que ejecutaba ese programa.
- El objeto `private` puede dejar restos. Los restos pueden ser el objeto RSA, la PEM en bytes, la PEM en texto y copias temporales o fragmentos de cualquiera de ellas.
- El buffer asociado a `keys.txt` también pudo dejar trazas.

En consecuencia, las posibles huellas en memoria no se limitaban a una clave PEM completa, sino que podían aparecer como:

- el objeto RSA;
- la PEM en bytes;
- la PEM en texto;
- copias temporales;
- o fragmentos parciales de cualquiera de ellas.

Buscar material PEM y Nombres sospechosos en disco

```
find /home/admin1 -type f \( -iname "*.pem" -o -iname "*.key" -o -iname
"*rsa*" -o -iname "*priv*" \) 2>/dev/null
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/__pycache__/test_RSA.cpython-36.pyc
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_RSA.py
/home/admin1/.local/lib/python3.6/site-packages/Crypto/PublicKey/_RSA.py
/home/admin1/.local/lib/python3.6/site-packages/Crypto/PublicKey/RSA.py
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/PublicKey/__pycache__/RSA.cpython-36.pyc
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/PublicKey/__pycache__/_RSA.cpython-36.pyc
/home/admin1/.local/bin/pyrsa-verify
/home/admin1/.local/bin/pyrsa-decrypt
/home/admin1/.local/bin/pyrsa-sign
/home/admin1/.local/bin/pyrsa-keygen
/home/admin1/.local/bin/pyrsa-priv2pub
/home/admin1/.local/bin/pyrsa-encrypt
```

```
grep -R -n "BEGIN RSA PRIVATE KEY\|BEGIN PRIVATE KEY\|END RSA PRIVATE
KEY\|END PRIVATE KEY" /home/admin1 2>/dev/null
/home/admin1/.bash_history:2:sudo grep -ra "BEGIN RSA PRIVATE KEY"
/home/admin2 2>/dev/null
/home/admin1/.bash_history:10:sudo grep -a -A 20 "BEGIN RSA PRIVATE KEY"
/dev/sda1
/home/admin1/crypy_decryptor.py:5:PRIV_KEY = b"""-----BEGIN RSA PRIVATE
KEY-----
/home/admin1/crypy_decryptor.py:7:-----END RSA PRIVATE KEY-----"""
Binary file /home/admin1/.local/lib/python3.6/site-
packages/rsa/__pycache__/pem.cpython-36.pyc matches
Binary file /home/admin1/.local/lib/python3.6/site-
packages/rsa/__pycache__/key.cpython-36.pyc matches
/home/admin1/.local/lib/python3.6/site-packages/rsa/pem.py:86:        when
your file has '-----BEGIN RSA PRIVATE KEY-----' and
/home/admin1/.local/lib/python3.6/site-packages/rsa/pem.py:87:        '----
-END RSA PRIVATE KEY-----' markers.
/home/admin1/.local/lib/python3.6/site-packages/rsa/pem.py:113:        when
your file has '-----BEGIN RSA PRIVATE KEY-----' and
/home/admin1/.local/lib/python3.6/site-packages/rsa/pem.py:114:        '---
--END RSA PRIVATE KEY-----' markers.
/home/admin1/.local/lib/python3.6/site-packages/rsa/key.py:566:        The
contents of the file before the "-----BEGIN RSA PRIVATE KEY-----" and
/home/admin1/.local/lib/python3.6/site-packages/rsa/key.py:567:
```

```
after the "-----END RSA PRIVATE KEY-----" lines is ignored.
Binary file /home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Cipher/__pycache__/test_pkcs1_15.cpython-36.pyc
matches
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Cipher/test_pkcs1_15.py:68:                '''----
-BEGIN RSA PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Cipher/test_pkcs1_15.py:82:-----END RSA PRIVATE
KEY-----''',
Binary file /home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Signature/__pycache__/test_pkcs1_15.cpython-36.pyc
matches
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Signature/test_pkcs1_15.py:110:
"""-----BEGIN RSA PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/Signature/test_pkcs1_15.py:118:                ---
--END RSA PRIVATE KEY-----""",
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:45:    rsaKeyPEM =
'''-----BEGIN RSA PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:53:-----END RSA
PRIVATE KEY-----'''
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:56:    rsaKeyPEM8 =
'''-----BEGIN PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:65:-----END PRIVATE
KEY-----'''
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:71:        ('test',
'''-----BEGIN RSA PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:82:-----END RSA
PRIVATE KEY-----''',
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:86:        ('rocking',
'''-----BEGIN RSA PRIVATE KEY-----
/home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/test_importKey.py:97:-----END RSA
PRIVATE KEY-----''',
Binary file /home/admin1/.local/lib/python3.6/site-
packages/Crypto/SelfTest/PublicKey/__pycache__/test_importKey.cpython-
36.pyc matches
```

```
admin1@mru8:~$ grep -R -n "YOUR PRIVATE KEY HERE" /home/admin1 2>/dev/null
/home/admin1/crypy_decryptor.py:6:YOUR PRIVATE KEY HERE
```

Conclusión: La búsqueda en disco no revela una clave privada válida almacenada de forma directa en `/home/admin1`. Las salidas que encontramos son

- Múltiples ficheros pertenecientes a librerías y utilidades criptográficas instaladas en el entorno (Crypto, rsa, pyrsa-*), que explican la presencia de material relacionado con RSA pero no constituyen por sí mismos la clave del ejercicio.
- El fichero `crypy_decryptor.py`, que ya sabemos que contiene únicamente un marcador de posición (YOUR PRIVATE KEY HERE).

Comprobar si crypy sigue vivo o dejó trazas simples

```
admin1@mru8:~$ ps aux | grep -i crypy | grep -v grep
admin1@mru8:~$ pgrep -a -f crypy
admin1@mru8:~$ lsof 2>/dev/null | grep -i crypy
```

Donde:

- Comprobamos que el proceso no está vivo.
- Comprobamos que el proceso no dejó trazas.

Revisar directorios temporales

```
admin1@mru8:~$ find /tmp /var/tmp /dev/shm -maxdepth 2 -type f -printf
"%TY-%Tm-%Td %TT %p\n" 2>/dev/null | sort
2026-03-20 17:16:05.1488950260 /tmp/region_988_heap.bin
2026-03-20 17:29:51.4697939700 /tmp/region_8596_big1.bin
```

Donde:

- No aparece nada interesante en los directorios temporales.

Revisar el entorno Python y las utilidades RSA instaladas

```
admin1@mru8:~$ which pyrsa-keygen pyrsa-encrypt pyrsa-decrypt pyrsa-
priv2pub 2>/dev/null
/home/admin1/.local/bin/pyrsa-keygen
/home/admin1/.local/bin/pyrsa-encrypt
/home/admin1/.local/bin/pyrsa-decrypt
/home/admin1/.local/bin/pyrsa-priv2pub
```

```
admin1@mru8:~$ python3 -c "import rsa,sys; print(rsa.__file__)"
/home/admin1/.local/lib/python3.6/site-packages/rsa/__init__.py
```

```
admin1@mru8:~$ python3 -c "import rsa; print(rsa.__version__)"
4.7
```

Conclusión: La revisión del entorno Python confirmó que en el sistema estaban instaladas tanto la librería rsa como varias utilidades asociadas (`pyrsa-keygen`, `pyrsa-encrypt`, `pyrsa-decrypt` y `pyrsa-priv2pub`). Además, se verificó que la implementación utilizada correspondía al paquete rsa versión 4.7, instalado en el entorno local del usuario. En consecuencia, el sistema dispone de todas las herramientas necesarias para crear, convertir y utilizar claves RSA.

Escaneo de la memoria física activa

```
admin1@mru8:~$ strings /tmp/* | grep -A 15 "BEGIN RSA PRIVATE KEY"
strings: Warning: '/tmp/systemd-private-3eac0455ac594507a3532ed960677c55-
systemd-resolved.service-j1f3hM' is a directory
strings: Warning: '/tmp/systemd-private-3eac0455ac594507a3532ed960677c55-
systemd-timesyncd.service-3ACQLm' is a directory
```

donde:

- Se realiza una búsqueda de cadenas en `/tmp` orientada a localizar bloques PEM de clave privada RSA. La prueba no aporta coincidencias.
- La prueba no aportó coincidencias útiles.
- Únicamente se obtuvieron advertencias debidas a la presencia de directorios privados de systemd en `/tmp`, que strings no procesa.

En consecuencia, no se recuperó material criptográfico útil en esta comprobación.

Escaneo de la memoria física

```
sudo strings /dev/mem | grep -A 20 "BEGIN RSA PRIVATE KEY"
```

donde:

- no devuelve nada.

Localización por desplazamiento - Offset

Si la memoria RAM ya ha sido sobrescrita, la clave podría estar en el espacio de intercambio (swap):

```
sudo grep -aob "BEGIN RSA PRIVATE KEY" /dev/mem
grep: /dev/mem: Operation not permitted
```

donde:

- El error `Operation not permitted` al intentar acceder a `/dev/mem` es común en kernels de Linux modernos, debido a la protección `CONFIG_STRICT_DEVMEM`, la cual impide que incluso el usuario root acceda directamente a toda la memoria física para proteger secretos del sistema.
- Sin embargo, como el enunciado indica que la clave está en memoria, existen rutas alternativas para extraer esos fragmentos: Usamermos `/proc/kcore` en vez de `/dev/mem`.

Localización por proc-kcore

```
admin1@mru8:~$ sudo strings /proc/kcore | grep -A 20 "BEGIN RSA PRIVATE
KEY"
```

donde:

- Esta vía resulta impracticable, ya que `/proc/kcore` representa una vista pseudo-ELF de la memoria virtual del kernel y su tamaño aparente es enorme.
- El escaneo completo no terminaba en un tiempo razonable.

Escaneo profundo de la partición de sistema

```
sudo strings /dev/sda1 | grep -E "^MII[A-Za-z0-9+/=]{40,}"
```

donde:

- No se obtuvieron resultados relevantes.

Lista de procesos

```
admin1@mru8:~$ ps aux | egrep 'crypy|python'
root       988  0.0  0.0 185952     0 ?        Ssl  15:03   0:00
/usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown
--wait-for-signal
root      8596  0.0  1.6 169520 16896 ?        Ssl  15:10   0:00
/usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
```

```
admin1@mru8:~$ sudo cat /proc/988/maps | grep rw-p | head
009b4000-00a51000 rw-p 003b4000 08:02 393301
/usr/bin/python3.6 (deleted)
00a51000-00a84000 rw-p 00000000 00:00 0
02045000-02256000 rw-p 00000000 00:00 0
[heap]
7f5dfc000000-7f5dfc021000 rw-p 00000000 00:00 0
7f5e01440000-7f5e01c40000 rw-p 00000000 00:00 0
7f5e01e5d000-7f5e01e5e000 rw-p 0001d000 08:02 917590
/lib/x86_64-linux-gnu/libudev.so.1.6.9 (deleted)
7f5e020d8000-7f5e020d9000 rw-p 0007a000 08:02 393453
/usr/lib/x86_64-linux-gnu/libzstd.so.1.3.3 (deleted)
7f5e022e8000-7f5e022e9000 rw-p 0000f000 08:02 918276
/lib/x86_64-linux-gnu/libbz2.so.1.0.4
7f5e02500000-7f5e02501000 rw-p 00017000 08:02 917573
/lib/x86_64-linux-gnu/libgcc_s.so.1
7f5e02884000-7f5e02886000 rw-p 00183000 08:02 409014
/usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.25

admin1@mru8:~$ sudo cat /proc/8596/maps | grep rw-p | head
009b4000-00a51000 rw-p 003b4000 08:02 393301
/usr/bin/python3.6 (deleted)
00a51000-00a84000 rw-p 00000000 00:00 0
012fe000-0150e000 rw-p 00000000 00:00 0
```

```
[heap]
7f488c000000-7f488c021000 rw-p 00000000 00:00 0
7f4890a97000-7f4891297000 rw-p 00000000 00:00 0
7f489149b000-7f489149c000 rw-p 00004000 08:02 395670
/usr/lib/python3/dist-packages/_dbus_glib_bindings.cpython-36m-x86_64-
linux-gnu.so
7f489149c000-7f48914dc000 rw-p 00000000 00:00 0
7f48916f0000-7f48916f1000 rw-p 00014000 08:02 918297
/lib/x86_64-linux-gnu/libgpg-error.so.0.22.0
7f4891a07000-7f4891a0c000 rw-p 00116000 08:02 917693
/lib/x86_64-linux-gnu/libgcrypt.so.20.2.1
7f4891a0c000-7f4891a0d000 rw-p 00000000 00:00 0
admin1@mru8:~$
```

Donde:

- El proceso crypy ya no está en ejecución, lo cual coincide con lo indicado en el enunciado.
- Los procesos de Python que vemos son servicios legítimos del sistema.

Buscamos los rastros que deja crypy

```
Select-String -Path "strings_output.txt" -Pattern "crypy" | ForEach-Object
{
    "Línea $($_.LineNumber): $($_.Line)"
}

Línea 1231246: crypy_decryptor.py
Línea 1231248: crypy.tx
Línea 1231250: .crypy_decryptor.py.swpncrypted
Línea 1452345: crypy_decryptor.py
Línea 1452347: crypy.tx
Línea 1452349: .crypy_decryptor.py.swpncrypted
Línea 3414308: crypy_decryptor.py
Línea 3414310: crypy.tx
Línea 3414312: .crypy_decryptor.py.swpncrypted
Línea 3429325: crypy
```

Donde:

- En el fichero `.crypy_decryptor.py.swp`: El sufijo .swp indica un archivo de intercambio (swap file) de Vim. Estos archivos se crean automáticamente cuando alguien edita un script. Lo más importante: los archivos swap suelen contener fragmentos del texto que se estaba editando en ese momento, incluyendo claves que se hayan pegado o hardcodeado.
- `crypy.tx`: Probablemente sea un fragmento de `crypy.txt o crypy.tx`.
- `ncrypted`: Es parte de la palabra encrypted.
- El proceso crypy deja rastros en tres bloques de memoria principales (rango 1.2M, 1.4M y el más importante en 3.4M).

Conclusión: Dado que estas líneas aparecen en tres bloques diferentes (1.2M, 1.4M y 3.4M), es muy probable que el sistema haya hecho volcados de memoria del editor en diferentes momentos. Para centrarnos en la zona de memoria donde aparece crypy, lo correcto es priorizar lo que está cerca de 3414308–3429325. Vamos a buscar y analizar las PEMs que aparezcan en esas zonas.

La pista de la Clave Fragmentada

Se buscaron indicios de claves privadas fragmentadas o almacenadas en formatos alternativos, como S-expressions:

```
Select-String -Path "strings_output.txt" -Pattern "\(\s*private-key\s+\
(rsa" -Context 0,8

> strings_output.txt:2642254: (private-key  (rsa  (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)   (e
#010001#)   (d
#07EF82500C403899934FE993AC5A36F14FF2DF38CF1EF315F205EE4C83EDAA19
8890FC23DE9AA933CAFB37B6A8A8DBA67541195833728731 0D3FF2F1DDC0CB93
7E70F57F75F833C021852B631D2B9A520E4431A03C5C3FCB5742DCD841D9FB12
771AA1620DCEC3F1583426066ED9DC3F7028C5B59202C88FDF20396E2FA0EC4F
5A22D9008F3043673931BC14A5046D6327398327900867E39CC61B2D1AFE2F48
EC8E1E3861C68D257D7425F4E6F99ABD77D61F10CA100EFC14389071831B33DD
69CC8EABEF860D1DC2AAA84ABEAE5DFC91BC124DAF0F4C8EF5BBEA436751DE84
3A8063E827A024466F44C28614F93B0732A100D4A0D86D532FE1E22C7725E401#)   (p
#00C29D438F115825779631CD665A5739367F3E128ADC29766483A46CA80897E0
79B32881860B8F9A6A04C2614A904F6F2578DAE13EA67CD60AE3D0AA00A1FF9B
441485E44B2DC3D0B60260FBFE073B5AC72FAF67964DE15C8212C389D20DB9CF
54AF6AEF5C4196EAA56495DD30CF709F499D5AB30CA35E086C2A1589D6283F1783#)   (q
#00D1984135231CB243FE959C0CBEF551EDD986AD7BEDF71EDF447BE3DA27AF46
79C974A6FA69E4D52FE796650623DE70622862713932AA2FD9F2EC856EAEAA77
88B4EA6084DC81C902F014829B18EA8B2666EC41586818E0589E18876065F97E
8D22CE2DA53A05951EC132DCEF41E70A9C35F4ACC268FFAC2ADF54FA1DA110B919#)   (u
#67CF0FD7635205DD80FA814EE9E9C267C17376BF3209FB5D1BC42890D2822A04
479DAF4D5B6ED69D0F8D1AF94164D07F8CD52ECEFE880641FA0F41DDAB1785E4
A37A32F997A516480B4CD4F6482B9466A1765093ED95023CA32D5EDC1E34CEE9
AF595BC51FE43C4BF810FA225AF697FB473B83815966188A4312C048B885E3F7#)))
  strings_output.txt:2642255:pgyx
  strings_output.txt:2642256:elg_testkey    => %s
  strings_output.txt:2642257:../../cipher/elgamal.c
  strings_output.txt:2642258:choosing a random k
  strings_output.txt:2642259:pk_elg
  strings_output.txt:2642260:elg_sign   data
  strings_output.txt:2642261:elg_sign      p
  strings_output.txt:2642262:elg_sign      g
> strings_output.txt:2939447:(key-data (public-key  (rsa(n%m)(e%m)))
(private-key  (rsa(n%m)(e%m)(d%m)(p%m)(q%m)(u%m))) %S)
  strings_output.txt:2939448:generate_std
```

```
  strings_output.txt:2939449:generate_fips
  strings_output.txt:2939450:gen_x931_parm_xi
  strings_output.txt:2939451:gen_x931_parm_xp
  strings_output.txt:2939452:generate_x931

strings_output.txt:2939453:18022e2593a402a737caaa93b4c7e750e20ca265452980e1
d6b7710fbd3e7dce72be5c2110fb47691cb38f42170ee3b4a37f2498d4a51567d762585e4cb
81d04fbc7df4144f8e5eac2d4b
8688521b64011f11d7ad53f4c874004819856f2e2a6f83d1c9c4e73ac26089789c14482b0b8
d44139133c88c4a52dba9dd6d6ffc622666b7d129168333d999706af30a2d7d272db7734e5e
dfb8c64ea3018af3ad20f4a013a5
060cb0f5e72753967bebe294280a6ed0ddbd3c4f11d0a8696e9d32a0dc03deb0b5e49b2cbd1
503392642d4e1211f3e8e2ee38abaa3671ccd57fcde8ca76e85fd2cb77c35706a970a213a27
352cec92a9604d543ddb5fc478ff
50e0622
  strings_output.txt:2939454:Jim quickly realized that the be
  strings_output.txt:2939455:`t1
```

Buscamos en el archivo de la memoria cualquier rastro de la clave privada RSA `MIIEowIBAAKCA…`.Se muestran las 6 líneas siguientes para que pueda ver el contexto.

```
Select-String -Path "strings_output.txt" -Pattern "BEGIN RSA PRIVATE
KEY|BEGIN PRIVATE KEY|MIIEowIBAAKCAQ|MIIEpAIBAAKCAQ" -Context 0,6

> strings_output.txt:3199317:            ------BEGIN PRIVATE KEY------
  strings_output.txt:3199318:            <key data>
  strings_output.txt:3199319:            ------END PRIVATE KEY-------
  strings_output.txt:3199320:utilc
  strings_output.txt:3199321:SaltConstantsz>
  strings_output.txt:3199322:    defines default distribution specific salt
variables
  strings_output.txt:3199323:    c
> strings_output.txt:5207080:-----BEGIN PRIVATE KEY-----
  strings_output.txt:5207081:-----BEGIN CERTIFICATE-----
  strings_output.txt:5207082:-----END CERTIFICATE-----
  strings_output.txt:5207083:GTlsCertificate
  strings_output.txt:5207084:Certificate (PEM)
  strings_output.txt:5207085:Private key
  strings_output.txt:5207086:Private key (PEM)
> strings_output.txt:5207090:-----BEGIN RSA PRIVATE KEY-----
  strings_output.txt:5207091:-----BEGIN ENCRYPTED PRIVATE KEY-----
  strings_output.txt:5207092:Cannot decrypt PEM-encoded private key
  strings_output.txt:5207093:No PEM-encoded private key found
  strings_output.txt:5207094:Could not parse PEM-encoded private key
  strings_output.txt:5207095:No PEM-encoded certificate found
  strings_output.txt:5207096:Could not parse PEM-encoded certificate
> strings_output.txt:5407166:            -----BEGIN RSA PRIVATE KEY-----
  strings_output.txt:5407167:
MIIBxwIBAAJhAKD0YSHy73nUgysO13XsJmd4fHiFyQ+00R7VVu2iV9Qco
  strings_output.txt:5407168:            ...
```

```
  strings_output.txt:5407169:            -----END RSA PRIVATE KEY-----
  strings_output.txt:5407170:        rsa_public: ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAGEAoPRhIfLvedSDKw7Xd ...
  strings_output.txt:5407171:        dsa_private: |
  strings_output.txt:5407172:            -----BEGIN DSA PRIVATE KEY-----
> strings_output.txt:5566208:-----BEGIN RSA PRIVATE KEY-----
>
strings_output.txt:5566209:MIIEpAIBAAKCAQEAz9xaWqRVjosGC5ifsNjuFmKf4fw+yctS
JbgttmjfH0IlkuTh

strings_output.txt:5566210:h+Vu/b7TBrzOCfGCnikdEMkBxREOm4gbaEyULsuV+/DF9doH
Ibl7IHo9vWiR1APG

strings_output.txt:5566211:0bdxa4gOw65fBn7k3MVG+tv4XFsjsub/5rj0TbUmh6+JiWTW
j+9AmUeKONriT5yv

strings_output.txt:5566212:Ypd8SRebpdVT82PcrXOfZNxqyssb4eODFRQM7bPGdmcsUdO9
rDQijozlluOad83j

strings_output.txt:5566213:b1djXNO+dyMFgpbKI1E5H6BnomUy6Bcz5rVME11hvJlXKl0p
ozxhjgcXlYSDKZt8

strings_output.txt:5566214:dawoupypKSN8b64dZE9pQxsj57JWTDy88XV3gwIDAQABAoIB
AFS0lHGJtH/xMZZ6

strings_output.txt:5566215:LOFxlZyztjnuhFvRqnlKk/5YwExJtWwmL64kllV7dR2yxTgS
Hkt7r6eOclUvfUdu
```

Nos vamos a quedar con las salidas que estén cerca de crypy y no con las que estén rodeadas de libgcrypt, dsa_testkey, ecc_testkey, elg_testkey.

Buscamos ciertos patrones y mostramos el contexto anterior y posterior:

```
Select-String -Path "strings_output.txt" -Pattern
"crypy_decryptor\.py|mysecrets\.txt\.encrypted|\.crypy_decryptor\.py\.swp|\
(\s*private-key\s+\(rsa|BEGIN RSA PRIVATE KEY|BEGIN PRIVATE KEY" -Context
40,80

  strings_output.txt:1231206:X>,
  strings_output.txt:1231207:;`o<
  strings_output.txt:1231208:libaccountsservice.so.0.0.0
...
  strings_output.txt:2642249:@^U
  strings_output.txt:2642250:autiful gowns are expensive.
  strings_output.txt:2642251:(data (flags pkcs1) (hash sha256
#112233445566778899 00aabbccddeeff8020304050607080 90a0b0c0d0f01121#))
  strings_output.txt:2642252:(data (flags pkcs1) (hash sha256
#112233445566778899 00aabbccddeeff1020304050607080 90a0b0c0d0f01121#))
  strings_output.txt:2642253: (public-key  (rsa   (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
```

```
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)   (e
#010001#)))
> strings_output.txt:2642254: (private-key  (rsa  (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)   (e
#010001#)   (d
#07EF82500C403899934FE993AC5A36F14FF2DF38CF1EF315F205EE4C83EDAA19
8890FC23DE9AA933CAFB37B6A8A8DBA67541195833728731 0D3FF2F1DDC0CB93
7E70F57F75F833C021852B631D2B9A520E4431A03C5C3FCB5742DCD841D9FB12
771AA1620DCEC3F1583426066ED9DC3F7028C5B59202C88FDF20396E2FA0EC4F
5A22D9008F3043673931BC14A5046D6327398327900867E39CC61B2D1AFE2F48
EC8E1E3861C68D257D7425F4E6F99ABD77D61F10CA100EFC14389071831B33DD
69CC8EABEF860D1DC2AAA84ABEAE5DFC91BC124DAF0F4C8EF5BBEA436751DE84
3A8063E827A024466F44C28614F93B0732A100D4A0D86D532FE1E22C7725E401#)   (p
#00C29D438F115825779631CD665A5739367F3E128ADC29766483A46CA80897E0
79B32881860B8F9A6A04C2614A904F6F2578DAE13EA67CD60AE3D0AA00A1FF9B
441485E44B2DC3D0B60260FBFE073B5AC72FAF67964DE15C8212C389D20DB9CF
54AF6AEF5C4196EAA56495DD30CF709F499D5AB30CA35E086C2A1589D6283F1783#)   (q
#00D1984135231CB243FE959C0CBEF551EDD986AD7BEDF71EDF447BE3DA27AF46
79C974A6FA69E4D52FE796650623DE70622862713932AA2FD9F2EC856EAEAA77
88B4EA6084DC81C902F014829B18EA8B2666EC41586818E0589E18876065F97E
8D22CE2DA53A05951EC132DCEF41E70A9C35F4ACC268FFAC2ADF54FA1DA110B919#)   (u
#67CF0FD7635205DD80FA814EE9E9C267C17376BF3209FB5D1BC42890D2822A04
479DAF4D5B6ED69D0F8D1AF94164D07F8CD52ECEFE880641FA0F41DDAB1785E4
A37A32F997A516480B4CD4F6482B9466A1765093ED95023CA32D5EDC1E34CEE9
AF595BC51FE43C4BF810FA225AF697FB473B83815966188A4312C048B885E3F7#)))
  strings_output.txt:2642255:pgyx
  strings_output.txt:2642256:elg_testkey    => %s
  strings_output.txt:2642257:../../cipher/elgamal.c
...
  strings_output.txt:2939438:extracting signature data failed
  strings_output.txt:2939439:signature does not match reference data

strings_output.txt:2939440:6252a19a11e1d5155ed9376036277193d644fa239397fff0
3e9b92d6f86415d6d30da9273775f290e580d038295ff8ff89522becccfa6ae870bf76b76df
402a854f69347e3db3de8e1e7d
4dada281ec556810c7a8ecd0b5f51f9b1c0e7aa755761aa2b8ba5f811304acc6af0eca41fe4
9baf33bf34eddaf44e21e036ac7f0b6803cdef1c60021fb7b5b97ebacdd88ab755ce29af568
dbc5728cc6e6eff42618d62a0386
ca8beed46402bdeeef29b6a3feded906bace411a06a39192bf516ae1067e4320fa8ea113968
525f4574d022a3ceeaafdc41079efe1f22cc94bf59d8d3328085da9674857db56de5978a623
94aab48aa3b72e23a1b16260cfd9
```

```
daafe65
  strings_output.txt:2939441:converting encrydata to mpi failed
  strings_output.txt:2939442:gcry_pk_decrypt returned garbage
  strings_output.txt:2939443:ciphertext doesn't match reference data
  strings_output.txt:2939444:_gcry_mpi_get_nbits ((xi)) == 101
  strings_output.txt:2939445:_gcry_mpi_get_nbits ((xp)) == nbits
  strings_output.txt:2939446:_gcry_mpi_gcd ( (g), (e), (phi) )
> strings_output.txt:2939447:(key-data (public-key  (rsa(n%m)(e%m)))
(private-key  (rsa(n%m)(e%m)(d%m)(p%m)(q%m)(u%m))) %S)
  strings_output.txt:2939448:generate_std
  strings_output.txt:2939449:generate_fips
  strings_output.txt:2939450:gen_x931_parm_xi
  strings_output.txt:2939451:gen_x931_parm_xp
  strings_output.txt:2939452:generate_x931

strings_output.txt:2939453:18022e2593a402a737caaa93b4c7e750e20ca265452980e1
d6b7710fbd3e7dce72be5c2110fb47691cb38f42170ee3b4a37f2498d4a51567d762585e4cb
81d04fbc7df4144f8e5eac2d4b
8688521b64011f11d7ad53f4c874004819856f2e2a6f83d1c9c4e73ac26089789c14482b0b8
d44139133c88c4a52dba9dd6d6ffc622666b7d129168333d999706af30a2d7d272db7734e5e
dfb8c64ea3018af3ad20f4a013a5
060cb0f5e72753967bebe294280a6ed0ddbd3c4f11d0a8696e9d32a0dc03deb0b5e49b2cbd1
503392642d4e1211f3e8e2ee38abaa3671ccd57fcde8ca76e85fd2cb77c35706a970a213a27
352cec92a9604d543ddb5fc478ff
50e0622
...
  strings_output.txt:3438046:p1w
  strings_output.txt:3438047:p1w
  strings_output.txt:3438048:{3c
  strings_output.txt:3438049:/home/admin1/script.py
  strings_output.txt:3438050:         43582 universe/debian-
installer/binary-s390x/Packages
  strings_output.txt:3438051: 30b0f42a1253abc0f7b47f310015867f331ca0df
13900 universe/debian-installer/binary-s390x/Packages.gz
  strings_output.txt:3438052: 54742223942600e8f340e03f0e6c5042ed7c886b
12688 universe/debian-installer/binary-s390x/Packages.xz
  strings_output.txt:3438053: cd506a7b7bf2bfb47b69a7c8f1680c4b301ab179
1440322 universe/dep11/Components-amd64.yml
  strings_output.txt:3438054: 13164d741203f7b0bb74571f17d6102e79202094
419964 universe/dep11/Components-amd64.yml.gz
  strings_output.txt:3438055: 2b7092e8f43e9e5e3cc794e03cb3714ac87b1055
302936 universe/dep11/Components-amd64.yml.xz
  strings_output.txt:3438056: 5f9056192afa06cd864e5e506b20fb0c8ad27310
1411735 universe/dep11/Components-arm64.yml
  strings_output.txt:3438057: 0dda776b0b86f33965a32efbee17cf43861a82a6
410538 universe/dep11/Components-arm64.yml.gz
  strings_output.txt:3438058: b24f704a457acf58530535efd3073b4060311208
297240 universe/dep11/Components-arm64.yml.xz
  strings_output.txt:3438059: 8bffb3efca16cb66566a80607b698c23fe959caa
1411735 universe/dep11/Components-armhf.yml
  strings_output.txt:3438060: 241ee1445f4ed0806812e8d75dc51800c07fa2ff
407891 universe/dep11/Components-armhf.yml.gz
  strings_output.txt:3438061: b867089e8162288e3a550193616f5cd9c3b6e7b2
297340 universe/dep11/Components-armhf.yml.xz
```

```
  strings_output.txt:3438062: be58fa248e22d14d2d6a5e4eb3159e28524a8774
1440322 universe/dep11/Components-i386.yml
  strings_output.txt:3438063: 8715b307662d5aab869682556d77a9b870c48ac5
418923 universe/dep11/Components-i386.yml.gz
  strings_output.txt:3438064: c7dbf5cb3fd83eaf2bc026320cf41922137dde3f
302576 universe/dep11/Components-i386.yml.xz
  strings_output.txt:3438065: 86ec4493dede4e569b3d800511314ba84a3cf3b3
1400709 universe/dep11/Components-ppc64el.yml
  strings_output.txt:3438066: f5e8658775f48c121513d9be4d52ea21c85cf2a4
404255 universe/dep11/Components-ppc64el.yml.gz
  strings_output.txt:3438067: 6da849d48e77cc00b7a386693227325bf9be2528
293468 universe/dep11/Components-ppc64el.yml.xz
  strings_output.txt:3438068: c273a8de856be80ce91a5559d43898ed352cb9ad
1396737 universe/dep11/Components-s390x.yml
  strings_output.txt:3438069: 915fee037c555ba569bad901269586876dbbcbd1
403487 universe/dep11/Components-s390x.yml.gz
  strings_output.txt:3438070: 8ab664d2e9d77a685a4e5e6a39c9e6fe9348e3f0
292688 universe/dep11/Components-s390x.yml.xz
  strings_output.txt:3438071: 00c849e81af672b8c73327061a1cb09ecdf2fe44
1456128 universe/dep11/icons-128x128.tar
  strings_output.txt:3438072: b24934281878c7dd928c4e42567b4bf115b2d339
1119039 universe/dep11/icons-128x128.tar.gz
  strings_output.txt:3438073: 60cacbf3d72e1e7834203da608037b1bf83b40e8
1024 universe/dep11/icons-128x128@2.tar
  strings_output.txt:5207040:Whether to use default fallbacks found by
shortening the name at
  strings_output.txt:5207041: characters. Ignores names after the first if
multiple names are given.
  strings_output.txt:5207042:themed->names != NULL && themed->names[0] !=
NULL
...
  strings_output.txt:5566205:|$H
  strings_output.txt:5566206:D$`t
  strings_output.txt:5566207:|$`
> strings_output.txt:5566208:-----BEGIN RSA PRIVATE KEY-----

strings_output.txt:5566209:MIIEpAIBAAKCAQEAz9xaWqRVjosGC5ifsNjuFmKf4fw+yctS
JbgttmjfH0IlkuTh

strings_output.txt:5566210:h+Vu/b7TBrzOCfGCnikdEMkBxREOm4gbaEyULsuV+/DF9doH
Ibl7IHo9vWiR1APG

strings_output.txt:5566211:0bdxa4gOw65fBn7k3MVG+tv4XFsjsub/5rj0TbUmh6+JiWTW
j+9AmUeKONriT5yv

strings_output.txt:5566212:Ypd8SRebpdVT82PcrXOfZNxqyssb4eODFRQM7bPGdmcsUdO9
rDQijozlluOad83j

strings_output.txt:5566213:b1djXNO+dyMFgpbKI1E5H6BnomUy6Bcz5rVME11hvJlXKl0p
ozxhjgcXlYSDKZt8

strings_output.txt:5566214:dawoupypKSN8b64dZE9pQxsj57JWTDy88XV3gwIDAQABAoIB
AFS0lHGJtH/xMZZ6
```

```
strings_output.txt:5566215:LOFxlZyztjnuhFvRqnlKk/5YwExJtWwmL64kllV7dR2yxTgS
Hkt7r6eOclUvfUdu

strings_output.txt:5566216:cNTRGDX1qvpPhSyKHAvnPUDf79cHDR5cJ6KA4gyTFokxvvGz
tmotTp6eE5j3XxXh

strings_output.txt:5566217:wvjv+EbOgpQaua6u1C01r5qHJ9HAzRnxnyLqC/Dhp0HWkXit
f7WbYc4yRHyUK6o9

strings_output.txt:5566218:j5DixH8rhxxIDFby5sBHtvU4Qd9JVyXXpPw7T61+K+FTHZA9
wSSlKP/T2KA7kDef

strings_output.txt:5566219:/WgbUlTDxHrAeLOgzEfuvggy4v34NDDhrlBiY1Wj9ZOXyMig
9F49hKY6O0HzlZVz

strings_output.txt:5566220:0ZYOEsECgYEA64OzvFqbyfJGpRpDRneRdfdUzJ+SgKkoKR2a
2d0Kto4IONls6dIY

strings_output.txt:5566221:tfEZsG327dl/ArDs5BYaoK4lygZjK7hbyBqHSSMMSZlHlx+f
gA4w1luih1YLA1lr

strings_output.txt:5566222:ipfUoAP2s52yycFRasFnHaFLhLMqkaLBE/LNrNU6jU3IaYhN
suyAWiMCgYEA4fDf

strings_output.txt:5566223:0ruElDjgAs9XeQLwfIR1AKuKl0uCX9r66jRN9NKZnbMSFVd4
G2rx4yQXkBYpiE3a

strings_output.txt:5566224:tgVLVNCwuLLmJVlbsNnTl+y8N/4NW1aDXiaEatPmluT/sNev
7sUOnHaRYXopOBuT

strings_output.txt:5566225:6fsyGHODgxzEJThc3IOnOduh528bPyl4F59V0yECgYEAw3l1
peDiqzP+tKxeqE83

strings_output.txt:5566226:mVzmskvDsiw0XCPpUehoKusqId08y7mIrwJlGw26ROIfzCED
DbDW+wRv8wVoLHKB

strings_output.txt:5566227:I035eZewbCnfxKwHm6arnE9ET+X3kBkY7Fhmr0V67sv2CAT/
SYcqyeoFHygCLgyT

strings_output.txt:5566228:CradRVVZmsyzifwK2XX4dlcCgYEAnlIuVLlHfqGX/wARWF+B
6o7aedy6YafstIR6

strings_output.txt:5566229:nFCIa8yDDikju8auB/BZjQOGa1XMRpHfdvqgvc76doINmRBT
msoYZfXiMg4Yh+9I

strings_output.txt:5566230:YFn3IfBYPVY8AUwyIMr+oQ7Icpiqd4GDlUqK4O1YszAeFcK3
92FddcJ8YfLOEeVa

strings_output.txt:5566231:HJtvBWECgYB1o0cyVOSoIhiHhIDB5Y+nSX4zO7b7OpyMAiQf
MnGUyeke482gdghz

strings_output.txt:5566232:qu8dq5ccK+zKdn7dafd83nWPHWyAxUzMZh5xTCMZIlJ5dLFn
IxeicF+EtzSxkZG7
```

```
strings_output.txt:5566233:N2bAuBowMH+IzVCbwyuMnH+GOR1G7R2+yVxh2Im1bokZ11Ym
unY0Rg==
  strings_output.txt:5566234:-----END RSA PRIVATE KEY-----
  strings_output.txt:5566235:scan the
  strings_output.txt:5566236: to read
…
```

Nos quedamos con la clave fragmentada en la zona del proceso crypy:

```
> strings_output.txt:2642254: (private-key  (rsa  (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)   (e
#010001#)   (d
#07EF82500C403899934FE993AC5A36F14FF2DF38CF1EF315F205EE4C83EDAA19
8890FC23DE9AA933CAFB37B6A8A8DBA67541195833728731 0D3FF2F1DDC0CB93
7E70F57F75F833C021852B631D2B9A520E4431A03C5C3FCB5742DCD841D9FB12
771AA1620DCEC3F1583426066ED9DC3F7028C5B59202C88FDF20396E2FA0EC4F
5A22D9008F3043673931BC14A5046D6327398327900867E39CC61B2D1AFE2F48
EC8E1E3861C68D257D7425F4E6F99ABD77D61F10CA100EFC14389071831B33DD
69CC8EABEF860D1DC2AAA84ABEAE5DFC91BC124DAF0F4C8EF5BBEA436751DE84
3A8063E827A024466F44C28614F93B0732A100D4A0D86D532FE1E22C7725E401#)   (p
#00C29D438F115825779631CD665A5739367F3E128ADC29766483A46CA80897E0
79B32881860B8F9A6A04C2614A904F6F2578DAE13EA67CD60AE3D0AA00A1FF9B
441485E44B2DC3D0B60260FBFE073B5AC72FAF67964DE15C8212C389D20DB9CF
54AF6AEF5C4196EAA56495DD30CF709F499D5AB30CA35E086C2A1589D6283F1783#)   (q
#00D1984135231CB243FE959C0CBEF551EDD986AD7BEDF71EDF447BE3DA27AF46
79C974A6FA69E4D52FE796650623DE70622862713932AA2FD9F2EC856EAEAA77
88B4EA6084DC81C902F014829B18EA8B2666EC41586818E0589E18876065F97E
8D22CE2DA53A05951EC132DCEF41E70A9C35F4ACC268FFAC2ADF54FA1DA110B919#)   (u
#67CF0FD7635205DD80FA814EE9E9C267C17376BF3209FB5D1BC42890D2822A04
479DAF4D5B6ED69D0F8D1AF94164D07F8CD52ECEFE880641FA0F41DDAB1785E4
A37A32F997A516480B4CD4F6482B9466A1765093ED95023CA32D5EDC1E34CEE9
AF595BC51FE43C4BF810FA225AF697FB473B83815966188A4312C048B885E3F7#)))
  strings_output.txt:2642255:pgyx
  strings_output.txt:2642256:elg_testkey    => %s
  strings_output.txt:2642257:../../cipher/elgamal.c
  strings_output.txt:2642258:choosing a random k
  strings_output.txt:2642259:pk_elg
  strings_output.txt:2642260:elg_sign   data
  strings_output.txt:2642261:elg_sign      p
  strings_output.txt:2642262:elg_sign      g
  strings_output.txt:2642263:elg_sign      y
  strings_output.txt:2642264:elg_sign      x
  strings_output.txt:2642265:elg_sign  sig_r
  strings_output.txt:2642266:elg_sign  sig_s
  strings_output.txt:2642267:(sig-val(elg(r%M)(s%M)))
```

```
  strings_output.txt:2642268:elg_sign      => %s
  strings_output.txt:2642269:elg_encrypt data
  strings_output.txt:2642270:pgy
  strings_output.txt:2642271:elg_encrypt  p
  strings_output.txt:2642272:elg_encrypt  g
  strings_output.txt:2642273:elg_encrypt  y
  strings_output.txt:2642274:(enc-val(elg(a%m)(b%m)))
> strings_output.txt:2939447:(key-data (public-key  (rsa(n%m)(e%m)))
(private-key  (rsa(n%m)(e%m)(d%m)(p%m)(q%m)(u%m))) %S)
  strings_output.txt:2939448:generate_std
  strings_output.txt:2939449:generate_fips
  strings_output.txt:2939450:gen_x931_parm_xi
  strings_output.txt:2939451:gen_x931_parm_xp
  strings_output.txt:2939452:generate_x931

strings_output.txt:2939453:18022e2593a402a737caaa93b4c7e750e20ca265452980e1
d6b7710fbd3e7dce72be5c2110fb47691cb38f42170ee3b4a37f2498d4a51567d762585e4cb
81d04fbc7df4144f8e5eac2d4b
8688521b64011f11d7ad53f4c874004819856f2e2a6f83d1c9c4e73ac26089789c14482b0b8
d44139133c88c4a52dba9dd6d6ffc622666b7d129168333d999706af30a2d7d272db7734e5e
dfb8c64ea3018af3ad20f4a013a5
060cb0f5e72753967bebe294280a6ed0ddbd3c4f11d0a8696e9d32a0dc03deb0b5e49b2cbd1
503392642d4e1211f3e8e2ee38abaa3671ccd57fcde8ca76e85fd2cb77c35706a970a213a27
352cec92a9604d543ddb5fc478ff
50e0622
  strings_output.txt:2939454:Jim quickly realized that the be
  strings_output.txt:2939455:`t1
  strings_output.txt:2939456:Po1
  strings_output.txt:2939457:Pp1
  strings_output.txt:2939458:0a1
  strings_output.txt:2939459:@p1
  strings_output.txt:2939460:`p1
  strings_output.txt:2939461:0p1
  strings_output.txt:2939462:`t1
  strings_output.txt:2939463:0`1
  strings_output.txt:2939464: S0
  strings_output.txt:2939465:0g1
  strings_output.txt:2939466:0g1
  strings_output.txt:2939467:P*1
```

Extraemos ese fragmento:

Get-Content "strings_output.txt" | Where-Object { $_.ReadCount -eq 2642254 }

```
(private-key  (rsa  (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)   (e
#010001#)   (d
#07EF82500C403899934FE993AC5A36F14FF2DF38CF1EF315F205EE4C83EDAA19
8890FC23DE9AA933CAFB37B6A8A8DBA67541195833728731 0D3FF2F1DDC0CB93
7E70F57F75F833C021852B631D2B9A520E4431A03C5C3FCB5742DCD841D9FB12
771AA1620DCEC3F1583426066ED9DC3F7028C5B59202C88FDF20396E2FA0EC4F
5A22D9008F3043673931BC14A5046D6327398327900867E39CC61B2D1AFE2F48
EC8E1E3861C68D257D7425F4E6F99ABD77D61F10CA100EFC14389071831B33DD
69CC8EABEF860D1DC2AAA84ABEAE5DFC91BC124DAF0F4C8EF5BBEA436751DE84
3A8063E827A024466F44C28614F93B0732A100D4A0D86D532FE1E22C7725E401#)   (p
#00C29D438F115825779631CD665A5739367F3E128ADC29766483A46CA80897E0
79B32881860B8F9A6A04C2614A904F6F2578DAE13EA67CD60AE3D0AA00A1FF9B
441485E44B2DC3D0B60260FBFE073B5AC72FAF67964DE15C8212C389D20DB9CF
54AF6AEF5C4196EAA56495DD30CF709F499D5AB30CA35E086C2A1589D6283F1783#)   (q
#00D1984135231CB243FE959C0CBEF551EDD986AD7BEDF71EDF447BE3DA27AF46
79C974A6FA69E4D52FE796650623DE70622862713932AA2FD9F2EC856EAEAA77
88B4EA6084DC81C902F014829B18EA8B2666EC41586818E0589E18876065F97E
8D22CE2DA53A05951EC132DCEF41E70A9C35F4ACC268FFAC2ADF54FA1DA110B919#)   (u
#67CF0FD7635205DD80FA814EE9E9C267C17376BF3209FB5D1BC42890D2822A04
479DAF4D5B6ED69D0F8D1AF94164D07F8CD52ECEFE880641FA0F41DDAB1785E4
A37A32F997A516480B4CD4F6482B9466A1765093ED95023CA32D5EDC1E34CEE9
AF595BC51FE43C4BF810FA225AF697FB473B83815966188A4312C048B885E3F7#)))
```

Donde:

- La salida muestra una una clave privada RSA completa en formato S-expression en la línea 2642254. La línea 2642253 es la pública. En la 2642254 están n, e, d, p, q y u, que son justo los componentes que necesitas para reconstruir una PEM.
- Clave pública: strings_output.txt:2642253
- Clave privada: strings_output.txt:2642254
- La privada empieza así:
  - `(private-key (rsa (n #...#) (e #010001#) (d #...#) (p #...#) (q #...#) (u #...#)))`
- Eso ya es la clave, pero no en PEM.

Guardamos la clave encontrada:

```
Get-Content "strings_output.txt" | Where-Object { $_.ReadCount -eq 2642254
} | Set-Content "rsa_private_sexpr.txt"

(private-key  (rsa  (n
#009F56231A3D82E3E7D613D59D53E9AB921BEF9F08A782AED0B6E46ADBC853EC
7C71C422435A3CD8FA0DB9EFD55CD3295BADC4E8E2E2B94E15AE82866AB8ADE8
7E469FAE76DC3577DE87F1F419C4EB41123DFAF8D16922D5EDBAD6E9076D5A1C
958106F0AE5E2E9193C6B49124C64C2A241C4075D4AF16299EB87A6585BAE917
DEF27FCDD165764D069BC18D16527B29DAAB549F7BBED4A7C6A842D203ED6613
6E2411744E432CD26D940132F25874483DCAEECDFD95744819CBCF1EA810681C
42907EBCB1C7EAFBE75C87EC32C5413EA10476545D3FC7B2ADB1B66B7F200918
664B0E5261C2895AA28B0DE321E921B3F877172CCCAB81F43EF98002916156F6CB#)
(e #010001#)   (d
#07EF82500C403899934FE993AC5A36F14FF2DF38CF1EF315F205EE4C83EDAA19
8890FC23DE9AA933CAFB37B6A8A8DBA67541195833728731 0D3FF2F1DDC0CB93
7E70F57F75F833C021852B631D2B9A520E4431A03C5C3FCB5742DCD841D9FB12
771AA1620DCEC3F1583426066ED9DC3F7028C5B59202C88FDF20396E2FA0EC4F
5A22D9008F3043673931BC14A5046D6327398327900867E39CC61B2D1AFE2F48
EC8E1E3861C68D257D7425F4E6F99ABD77D61F10CA100EFC14389071831B33DD
69CC8EABEF860D1DC2AAA84ABEAE5DFC91BC124DAF0F4C8EF5BBEA436751DE84
3A8063E827A024466F44C28614F93B0732A100D4A0D86D532FE1E22C7725E401#)
(p #00C29D438F115825779631CD665A5739367F3E128ADC29766483A46CA80897E0
79B32881860B8F9A6A04C2614A904F6F2578DAE13EA67CD60AE3D0AA00A1FF9B
441485E44B2DC3D0B60260FBFE073B5AC72FAF67964DE15C8212C389D20DB9CF
54AF6AEF5C4196EAA56495DD30CF709F499D5AB30CA35E086C2A1589D6283F1783#)
(q #00D1984135231CB243FE959C0CBEF551EDD986AD7BEDF71EDF447BE3DA27AF46
79C974A6FA69E4D52FE796650623DE70622862713932AA2FD9F2EC856EAEAA77
88B4EA6084DC81C902F014829B18EA8B2666EC41586818E0589E18876065F97E
8D22CE2DA53A05951EC132DCEF41E70A9C35F4ACC268FFAC2ADF54FA1DA110B919#)
(u #67CF0FD7635205DD80FA814EE9E9C267C17376BF3209FB5D1BC42890D2822A04
479DAF4D5B6ED69D0F8D1AF94164D07F8CD52ECEFE880641FA0F41DDAB1785E4
A37A32F997A516480B4CD4F6482B9466A1765093ED95023CA32D5EDC1E34CEE9
AF595BC51FE43C4BF810FA225AF697FB473B83815966188A4312C048B885E3F7#)))
```

Donde:

- n = módulo
- e = exponente público
- d = exponente privado
- p, q = primos RSA
- u = coeficiente CRT del formato usado por esa librería
- Tenemos lo necesario para reconstruir la PEM.

Usamos un programa Python para reconstruir la PEM: sexpr_a_pem.py. Obtenemos:


```
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAn1YjGj2C4+fWE9WdU+mrkhvvnwingq7QtuRq28hT7HxxxCJDWjzY+g2579V
c
0ylbrcTo4uK5ThWugoZquK3ofkafrnbcNXfeh/H0GcTrQRI9+vjRaSLV7brW6QdtWhyVgQbwrl4
u
kZPGtJEkxkwqJBxAddSvFimeuHplhbrpF97yf83RZXZNBpvBjRZSeynaq1Sfe77Up8aoQtID7WY
T
biQRdE5DLNJtlAEy8lh0SD3K7s39lXRIGcvPHqgQaBxCkH68scfq++dch+wyxUE+oQR2VF0/x7K
t
sbZrfyAJGGZLDlJhwolaoosN4yHpIbP4dxcszKuB9D75gAKRYVb2ywIDAQABAoIBAAfvglAMQDi
Z
k0/pk6xaNvFP8t84zx7zFfIF7kyD7aoZiJD8I96aqTPK+ze2qKjbpnVBGVgzcocxDT/y8d3Ay5N
+
cPV/dfgzwCGFK2MdK5pSDkQxoDxcP8tXQtzYQdn7EncaoWINzsPxWDQmBm7Z3D9wKMW1kgLIj98
g
OW4voOxPWiLZAI8wQ2c5MbwUpQRtYyc5gyeQCGfjnMYbLRr+L0jsjh44YcaNJX10JfTm+Zq9d9Y
f
EMoQDvwUOJBxgxsz3WnMjqvvhg0dwqqoSr6uXfyRvBJNrw9MjvW76kNnUd6EOoBj6CegJEZvRMK
G
FPk7BzKhANSg2G1TL+HiLHcl5AECgYEAwp1DjxFYJXeWMc1mWlc5Nn8+EorcKXZkg6RsqAiX4Hm
z
KIGGC4+aagTCYUqQT28leNrhPqZ81grj0KoAof+bRBSF5Estw9C2AmD7/gc7Wscvr2eWTeFcghL
D
idINuc9Ur2rvXEGW6qVkld0wz3CfSZ1aswyjXghsKhWJ1ig/F4MCgYEA0ZhBNSMcskP+lZwMvvV
R
7dmGrXvt9x7fRHvj2ievRnnJdKb6aeTVL+eWZQYj3nBiKGJxOTKqL9ny7IVurqp3iLTqYITcgck
C
8BSCmxjqiyZm7EFYaBjgWJ4Yh2Bl+X6NIs4tpToFlR7BMtzvQecKnDX0rMJo/6wq31T6HaEQuRk
C
gYEAlDKRqba5ev6P3yjcklrtdHR83j3j3acMb7qcC3FPxl6mcWgAnXkjSyyIoPHY5sRqspdrAvx
g
/9WXGeT7SmGM6NEADJK7LOwYTcNw4Ua8iio4Q8gMCw5cFakYQjE0xpP9lv9FeulvdQuxhvXDH5R
R
mAs9cC9lkEu1om54Yh6PvOUCgYAEFyOWpOPHeto6EzyQcXsy2B7vvc5sFzcmQAC3XjTmfDP5SdT
z
QhsC/1MJD7Cdg7X2sc0h36JAotFB2mzJjdwdV+KHFbCz+7dJ9iyCWF8jbhFuJOxTw8CFGjjCU5h
J
4TVh64IqilJXka0VkAg/UgH8AHWQ9Y5VBwgjKaZj86O+6QKBgGI5nCjcegHU0EKPH+sC5LyI29+
P
WFSxRaEOdLlScAKjNaUcD8jFVwGwGKmDAn2bbHhrMrDOLjnAKnejSgUwGCdx5gN7MZCMiieLlXN
4
lOI6417V4T8by66NtUlXlwZTyXnqrlk2BUWiiLmnoO4jtrk//svRq1ZNB3MDAB492AN/
-----END RSA PRIVATE KEY-----
```


Probamos a desencriptar el fichero pero se produce un error. Esta no es la clave correcta.

Repetimos el proceso para oros fragmentos y claves, pero ninguna sirve.

Encontrando diversas claves PEM en la memoria

```
Select-String -Path "strings_output.txt" -Pattern "^MII" |
    Where-Object { $_.Line.Length -gt 60 }
```

donde:

- Fichero obtenido: strings_crypy_comienzan_MII.txt.
- Al analizar las líneas que comienzan por MII, podemos identificar qué tipo de objetos criptográficos son
- Algunos fragmentos de interes que están la zona de memoria del proceso crypy:
  - Línea 5566209: `MIIEpAIBAAKCAQEAz9...`
  - Línea 3427144: Termina con `...Mo5qxN7SN5ShLHZ4swrhovO0C7jE=`. El símbolo = es relleno Base64, lo que indica que es el final de un fragmento o de una clave completa.
  - Línea 3531431: `MIIBIjANBgkqhkiG9w0BAQEFAAO...`. Este es el encabezado exacto de una Clave Pública RSA.

Algunas de las claves encontradas y analizadas:

- Candidata1
- Candidata2
- ....

En una fase inicial del análisis se localizaron diversas claves PEM y fragmentos con apariencia de clave RSA en el volcado de memoria RAM. Estas claves se consideraron candidatas potenciales para descifrar el fichero `mysecrets.txt.encrypted`.

Para verificar su validez, se reconstruyeron varias PEM completas o parciales y se probaron directamente contra el fichero cifrado. Aunque algunas de estas claves podían cargarse correctamente como claves RSA válidas, el proceso de descifrado devolvía errores, lo que indicaba que no correspondían a la clave realmente utilizada por el programa.

Un ejemplo del error obtenido fue el siguiente:

```
Traceback (most recent call last):
  File "...\crypy_decryptor.py", line 33, in <module>
    decrypted = rsa.decrypt(b64decode(f.read()), private_key)
  File "...site-packages\rsa\pkcs1.py", line 282, in decrypt
    raise DecryptionError("Decryption failed")
rsa.pkcs1.DecryptionError: Decryption failed
```

Estos resultados permiten concluir que las claves encontradas inicialmente en la RAM eran artefactos ajenos al flujo real del ejercicio, o restos de otras bibliotecas y entornos del sistema. Esta fase de prueba y error fue importante para reorientar el análisis hacia el artefacto principal `crypy` y, posteriormente, hacia la recuperación de `keys.txt` desde la swap.

Cambio de Estrategia: Recuperar crypy

Ya no vamos a buscar más claves PEMs en RAM. Vamos a recuperar o reconstruir el script `crypy`.

Volcado de Memoria RAM con AVML

```
wget https://github.com/microsoft/avml/releases/download/v0.14.0/avml
chmod +x avml
sudo ./avml ~/evidencia/ram.lime
```

Donde:

- Obtenemos un volcado de la memoria al fichero: `ram.lime`.

Copia de la swap

Como el proceso ya no existe, se obtiene también una copia de la swap, ya que puede contener restos del proceso, claves o ficheros borrados

```
sudo cp /swapfile ~/evidencia/swap.img
```

Extracción de cadenas imprimibles

Sobre la RAM y la swap se generan ficheros de cadenas con strings para facilitar la búsqueda de artefactos útiles:

```
strings -a -n 20 ~/evidencia/ram.lime > ~/evidencia/ram.lime.strings
strings -a -n 20 ~/evidencia/swap.img > ~/evidencia/swap.strings
```

Búsqueda de artefactos relevantes

Se hicieron búsquedas dirigidas sobre las evidencias para localizar:

- Nombres de ficheros del ejercicio (`crypy`, `crypy_decryptor.py`, `mysecrets.txt.encrypted`).
- Cabeceras `PEM` (`BEGIN RSA PRIVATE KEY`, `BEGIN RSA PUBLIC KEY`).
- Fragmentos `Base64` y restos de claves.

```
grep -RnaE 'crypy|mysecrets|encrypted|\.swp' ~/analisis/trabajo/evidencia
grep -RnaE 'BEGIN RSA PRIVATE KEY|BEGIN RSA PUBLIC KEY|AQABAoIB|CgYEA'
~/analisis/trabajo/evidencia
```

Buscamos si keys.txt existe

```
find /home/admin1 /tmp /var/tmp -type f -name 'keys.txt' 2>/dev/null
```

Donde:

- Vemos que `keys.txt` NO existe.

##**Sacamos un contexto amplio en swap alrededor de crypy**

```
sed -n '83770,84430p' ~/analisis/trabajo/swap.strings >
~/analisis/swap_crypy_ctx1.txt
sed -n '106560,107950p' ~/analisis/trabajo/swap.strings >
~/analisis/swap_crypy_ctx2.txt
sed -n '109100,109320p' ~/analisis/trabajo/swap.strings >
~/analisis/swap_crypy_ctx3.txt
```

Donde:

- Obtenemos: swap_crypy_ctx1.txt
- Obtenemos: swap_crypy_ctx2.txt
- Obtenemos: analisis/swap_crypy_ctx3.txt

Además, el fichero importante parece ser:

- `crypy` → 2516 bytes
- `crypy_decryptor.py` → 393 bytes

Esto sugiere que la lógica real está en `crypy`, no en `crypy_decryptor.py`.

Recuperamos el marcador Keys.txt de la Swap

Busca todas las menciones:

```
grep -Rna 'keys.txt' ~/analisis/trabajo/evidencia/*.strings
~/analisis/trabajo/evidencia/**/*.strings 2>/dev/null

/home/admin1/analisis/trabajo/evidencia/ram.lime.strings.strings:177649:#
https://github.com/torvalds/linux/blob/master/Documentation/security/keys.t
xt
/home/admin1/analisis/trabajo/evidencia/ram.lime.strings.strings:718829:
XML/HTML.  If you have the appropriate "keys.txt" file with your private
/home/admin1/analisis/trabajo/evidencia/ram.lime.strings.strings:4636113:#
https://github.com/torvalds/linux/blob/master/Documentation/security/keys.t
xt
/home/admin1/analisis/trabajo/evidencia/ram.lime.strings.strings:4876762:#
https://github.com/torvalds/linux/blob/master/Documentation/security/keys.t
xt
```

Buscamos exactamente en swap o RAM por `keys.txt`, `-----BEGIN RSA PRIVATE KEY-----`, `-----BEGIN PUBLIC KEY-----`, `public_pem`, `private_pem`:

```
grep -RnaE 'keys.txt|BEGIN RSA PRIVATE KEY|BEGIN PUBLIC KEY|BEGIN RSA
PUBLIC KEY|public_pem|private_pem' ~/analisis/trabajo/evidencia/*.strings
~/analisis/trabajo/evidencia/**/*.strings 2>/dev/null | tee
~/analisis/hits_keys.txt

....
....
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:88379:
These files can be recognised in that they start with BEGIN PUBLIC KEY
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:88380:
rather than BEGIN RSA PUBLIC KEY.
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:88381:
The contents of the file before the "-----BEGIN PUBLIC KEY-----" and
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:90206:
when your file has '-----BEGIN RSA PRIVATE KEY-----' and
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:92261:
The contents of the file before the "-----BEGIN RSA PRIVATE KEY-----" and
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:92770:
The contents of the file before the "-----BEGIN RSA PUBLIC KEY-----" and
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:95512:
when your file has '-----BEGIN RSA PRIVATE KEY-----' and
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:97132:-----
BEGIN RSA PUBLIC KEY-----
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:97694:-----
```

```
BEGIN RSA PUBLIC KEY-----
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:99294:-----
BEGIN RSA PUBLIC KEY-----
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:99302:-----
BEGIN RSA PRIVATE KEY-----
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:99372:-----
BEGIN RSA PRIVATE KEY-----
/home/admin1/analisis/trabajo/evidencia/swap.strings.strings:99393:-----
BEGIN RSA PRIVATE KEY-----
```

donde:

- Obtenemos el fichero: hits_keys.txt.
- Lo más prometedor de esta salida en swap:
  - `97132: -----BEGIN RSA PUBLIC KEY-----`
  - `97694: -----BEGIN RSA PUBLIC KEY-----`
  - `99294: -----BEGIN RSA PUBLIC KEY-----`
  - `99302: -----BEGIN RSA PRIVATE KEY-----`
  - `99372: -----BEGIN RSA PRIVATE KEY-----`
  - `99393: -----BEGIN RSA PRIVATE KEY-----`
  - **Esto sí se parece a restos de un `keys.txt` o de varias copias parciales de las claves.**

Sacamos esa ventana exacta de swap

Obtenemos swap_keys_region.txt

```
sed -n '99280,99430p' ~/analisis/trabajo/evidencia/swap.strings.strings >
~/analisis/swap_keys_region.txt
```

Donde:

- Utilizaremos este extracto de la swap en los siguientes comandos.

Buscamos restos del .pyc

```
grep -nE 'crypy\.cpython-36\.pyc|__pycache__/crypy|compileall crypy\.py'
~/analisis/trabajo/evidencia/swap.strings.strings


79527:python -m compileall crypy.py
79561:mv __pycache__/crypy.cpython-36.pyc .
79658:python3 crypy.cpython-36.pyc
79744:python3 -m compileall crypy.py
79780:python3 -m compileall crypy.py
79847:python3 __pycache__/crypy.cpython-36.pyc
79880:python3 crypy.cpython-36.pyc
79881:mv crypy.cpython-36.pyc crypy
107062:mv __pycache__/crypy.cpython-36.pyc .
109471:[00m$  compileall crypy.py : ~
```

donde:

- En `79881: mv crypy.cpython-36.pyc crypy`:
  - Se compiló `crypy.py` a bytecode.
  - Se sacó `crypy.cpython-36.pyc` de `__pycache__`.
  - Y se le renombró a `crypy`.
- Conclusión: `crypy` probablemente no era un script `.py`, sino un `.pyc` renombrado.

Recordando el fichero crypy

Ya inicimos al principio de este ejercicio un análisis profundo de este fichero. Recordamos algunas cosas interesantes:

```
file /home/admin1/crypy
/home/admin1/crypy: python 3.6 byte-compiled
admin1@mru8:~/analisis$ xxd -l 16 /home/admin1/crypy
00000000: 330d 0d0a f031 0660 1e0d 0000 e300 0000  3....1.`........
admin1@mru8:~/analisis$ strings -a -n 8 /home/admin1/crypy | head -50
PKCS1_OAEP)
PKCS1_v1_5)
    b64encode
    b64decode
SHA-256c
generate
    publickey)
keysizeZ
random_generator
crypy.py
```

```
    importKey)
Z   externKeyr
priv_keyr
...
...
Z   signaturer
./mysecrets.txt
keys.txt
./mysecrets.txt.encrypted
save_pkcs1
remove) Z
public_pemZ
private_pem
fZ  encryptedZ
encrypted_filer
__main__i?B
Crypto.PublicKeyr
Crypto.Cipherr
Crypto.Signaturer
Crypto.Hashr
__name__Z
<module>
```

Este fichero resulta que es un `bytecode Python 3.6`. Algunas cadenas que hemos sacado apuntan a la lógica real del ejercicio:

- `PKCS1_OAEP`
- `PKCS1_v1_5`
- `b64encode / b64decode`
- `priv_key`
- `publickey`
- `keys.txt`
- `./mysecrets.txt.encrypted`
- `save_pkcs1`
- `public_pem`
- `private_pem`

Eso sugiere que la clave probablemente no estaba en la RAM, sino que el propio `crypy` indica cómo la genera, guarda o carga. El siguiente paso que vamos a dar es desensamblar `crypy`.

Desensamblamos el fichero crypy

```
cp /home/admin1/crypy ~/analisis/crypy.cpython-36.pyc
python3 -m pip install xdis
pydisasm ~/analisis/crypy.cpython-36.pyc > ~/analisis/crypy.dis
```

Donde:

- Hacemos una copia con extensión `.pyc`.
- Usamos `xdis` para desensamblarlo.
- Obtenemos `crypy.dis`: crypy.dis

Filtramos el fichero `crypy.dis`:

```
grep -nE
'keys.txt|remove|save_pkcs1|open|write|encrypted_file|private_pem|public_pe
m' ~/analisis/crypy.dis

587:#    4: 'keys.txt'
593:#    0: open
597:#    4: save_pkcs1
598:#    5: write
603:#   10: remove
605:#   msg1, keysize, public, private, public_pem, private_pem, f,
encrypted, encrypted_file
611:#    4: public_pem
612:#    5: private_pem
615:#    8: encrypted_file
616: 82:           0 LOAD_GLOBAL          (open)
636:              34 LOAD_ATTR            (save_pkcs1)
638:              38 STORE_FAST           (public_pem)
641:              42 LOAD_ATTR            (save_pkcs1)
643:              46 STORE_FAST           (private_pem)
645: 91:          48 LOAD_GLOBAL          (open)
646:              50 LOAD_CONST           ("keys.txt")
653:              62 LOAD_ATTR            (write)
654:              64 LOAD_FAST            (public_pem)
661:              76 LOAD_ATTR            (write)
667:              86 LOAD_ATTR            (write)
668:              88 LOAD_FAST            (private_pem)
688:114:         124 LOAD_GLOBAL          (open)
693:             134 STORE_FAST           (encrypted_file)
695:115:         136 LOAD_FAST            (encrypted_file)
696:             138 LOAD_ATTR            (write)
707:             158 LOAD_ATTR            (remove)
```

Vemos algo del fichero desensamblado:

```
sed -n '700,730p' ~/analisis/crypy.dis
              146 POP_BLOCK
              148 LOAD_CONST           (None)
         >>   150 WITH_CLEANUP_START
              152 WITH_CLEANUP_FINISH
              154 END_FINALLY

117:          156 LOAD_GLOBAL          (os)
              158 LOAD_ATTR            (remove)
              160 LOAD_CONST           ("./mysecrets.txt")
              162 CALL_FUNCTION        (1 positional argument)
              164 POP_TOP
              166 LOAD_CONST           (None)
              168 RETURN_VALUE
```

donde:

- Aunque en las strings aparezcan `PKCS1_OAEP` y `PKCS1_v1_5`, el flujo principal que resuelve el ejercicio usa la librería `RSA`.
- Tiene una función `importKey`.
- Genera o maneja un par de claves RSA y guarda:
  - `public_pem`.
  - `private_pem`.
- Escribe esas claves en un archivo llamado `keys.txt`.
- El fichero cifrado de salida es `./mysecrets.txt.encrypted`.
- El `ciphertext` se guarda en Base64 con `b64encode`.
- `86: rsa.newkeys(2048)`: Genera un par nuevo.
- `91: open("keys.txt","w")`: Guarda `public_pem` y `private_pem`.
- `99: b64encode(rsa.encrypt(..., private))`.
- `114: open("./mysecrets.txt.encrypted","wb")`: Cifra y crea `mysecrets.txt.encrypted`.
- `117: os.remove("./mysecrets.txt")`: Borra `./mysecrets.txt`, no `keys.txt`.

Conclusión: la clave correcta no es la PEM repetida que vimos en la RAM, sino la que `crypy`escribe en `keys.txt`, y lo más probable es que sus restos estén en swap.

Buscamos cabeceras, pies y Base64 dentro de swap_keys_region.txt

```
grep -nE 'BEGIN|END|^[A-Za-z0-9+/]{20,}={0,2}$'
~/analisis/swap_keys_region.txt
6:tzinfotnqbL4Iwpn1LNPz7R
7:9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
8:/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
9:OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
10:Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
11:xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
12:93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
13:mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
14:-----END RSA PRIVATE KEY---
15:-----BEGIN RSA PUBLIC KEY-----
16:MIIBCgKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9/1JZ
17:1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU24LS
18:0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGkY8kA
19:jHOre6a1tRLNeLZZ5Tx5xY9JUOvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst3hse
20:EFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI6VrJFMLga/2HufDtaqpBgXMy
21:NCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQAB
22:-----END RSA PUBLIC KEY-----
23:-----BEGIN RSA PRIVATE KEY-----
24:MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9
25:/1JZ1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU
26:Bnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
27:Y8kAjHOre6a1tRLNeLZZ5Tx5xY9JUOvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst
28:3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI6VrJFMLga/2HufDtaqpB
29:gXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACoQnHhS5NYGcH32
30:YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69xoU
31:TguJN4xSPhTipij4LRXP39OGCpDj9HEtTyi7vwPrQxc0SQrfEf22Dj3XXJQquVxA
32:0iG3MwJMzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLT
33:m5433oxHFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasu
34:PaW93UZ1UsTLj9B/aAxz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6C
35:Xtqd4aECgYkArSVAU4F/SBUIZUPr1Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGc
36:M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
37:KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
38:tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
39:5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
40:SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
41:Y416AAJ0QpgIBriHie/9XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R
42:9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
43:/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
44:OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
45:Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
46:xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
47:93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
48:mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
49:-----END RSA PRIVATE KEY-----
53:CzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLT
```

```
54:m5433oxHFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasu
55:PaW93UZ1UsTLj9B/aAxz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6C
56:Xtqd4aECgYkArSVAU4F/SBUIZUPr1Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGc
57:M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
58:KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
59:tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
60:5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
61:SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
62:Y416AAJ0QpgIBriHie/9XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R
63:9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
64:/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
65:OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
66:Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
67:xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
68:93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
69:mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
70:-----END RSA PRIVATE KEY-----
72:gXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACoQnHhS5NYGcH32
73:YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69xoU
74:TguJN4xSPhTipij4LRXP39OGCpDj9HEtTyi7vwPrQxc0SQrfEf22Dj3XXJQquVxA
75:0iG3MwJMzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLT
76:m5433oxHFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasu
77:PaW93UZ1UsTLj9B/aAxz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6C
78:Xtqd4aECgYkArSVAU4F/SBUIZUPr1Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGc
79:M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
80:KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
81:tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
82:5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
83:SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
84:Y416AAJ0QpgIBriHie/9XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R
85:9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
86:/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
87:OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
88:Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
89:xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
90:93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
91:mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
92:-----END RSA PRIVATE KEY---
93:-----BEGIN RSA PRIVATE KEY-----
94:MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9
95:/1JZ1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU
96:24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
97:Y8kAjHOre6a1tRLNeLZZ5Tx5xY9JUOvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst
98:3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI6VrJFMLga/2HufDtaqpB
99:gXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACoQnHhS5NYGcH32
100:YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69xoU
101:TguJN4xSPhTipij4LRXP39OGCpDj9HEtTyi7vwPrQxc0SQrfEf22Dj3XXJQquVxA
102:0iG3MwJMzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLT
103:m5433oxHFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasu
104:PaW93UZ1UsTLj9B/aAxz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6C
105:Xtqd4aECgYkArSVAU4F/SBUIZUPr1Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGc
106:M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
107:KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
108:tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
```

```
109:5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
110:SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
112:MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9/1JZ1rv
M9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU24LS0XY7IXmuh3e4yO
mBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGkY8kAjHOre6a1tRLNeLZZ5Tx5xY9JU
OvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXg
WorI6VrJFMLga/2HufDtaqpBgXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACo
QnHhS5NYGcH32YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69x
oUTguJN4xSPhTipij4LRXP39OGq
113:0iG3MwJMzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLTm5433ox
HFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasuPaW93UZ1UsTLj9B/aA
xz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6CXtqd4aECgYkArSVAU4F/SBUIZUPr1
Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGcM7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5
XVxeC4CkKcgRRxKPwHWsw4skKZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E
+hdD1Z2Y8CDBLtQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrH
ni5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1SYnMyEWNz
6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIYY416AAJ0QpgIBriHie/9
XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R9D382QToU7yhKx5gzFdEgHN3MjINTN8
pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYj
HMZfkyON3E80m1P0+eNq5wOlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3
EBZZlS7H+rgZ48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF93Ggkjhm98H
PJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLskmMCG3O2nvua9qPD4q2EF+I
vhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
114:-----BEGIN RSA PRIVATE KEY-----
115:MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9
116:/1JZ1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU
117:24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
118:Y8kAjHOre6a1tRLNeLZZ5Tx5xY9JUOvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst
119:3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI6VrJFMLga/2HufDtaqpB
120:gXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACoQnHhS5NYGcH32
121:YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69xoU
122:TguJN4xSPhTipij4LRXP39OGCpDj9HEtTyi7vwPrQxc0SQrfEf22Dj3XXJQquVxA
123:1z8+TW3/yjCOWW2O0f0SGc
124:M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
125:KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
126:tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
127:5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
128:SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
129:Y416AAJ0QpgIBriHie/9XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R
130:9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
131:/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
132:OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
133:Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
134:xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
135:93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
136:mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
137:-----END RSA PRIVATE KEY-----
138:MIIBCgKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9/1JZ1rvM9KV
iZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU24LS0XY7IXmuh3e4yOmBnl
2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGkY8kAjHOre6a1tRLNeLZZ5Tx5xY9JUOvRh
xUdjAc5fYvDn6N2SaIcsO5H3P08XYst3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI
6VrJFMLga/2HufDtaqpBgXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQAB
```

donde:

- **La copia buena parece ser la de swap_keys_region.txt en las líneas 23–49.**
- Las demás copias están truncadas o corruptas:
  - línea 14: `-----END RSA PRIVATE KEY---` → rota
  - líneas 53–70: fragmento sin cabecera
  - líneas 72–92: otra copia parcial, también rota
  - líneas 93–137: parece casi completa, pero en la 123 ya se ve truncación
- La de 23–49 es la mejor porque tiene:
  - `-----BEGIN RSA PRIVATE KEY-----`
  - `cuerpo Base64 continuo`
  - `-----END RSA PRIVATE KEY-----`

Obtenemos una clave candidata

Obtenemos: candidata_swap_private.pem

```
sed -n '23,49p' ~/analisis/swap_keys_region.txt >
~/analisis/candidata_swap_private.pem
openssl rsa -in ~/analisis/candidata_swap_private.pem -check -noout
unable to load Private Key
140206130348480:error:09091064:PEM routines:PEM_read_bio_ex:bad base64
decode:../crypto/pem/pem_lib.c:929:
```

donde:

- Confirmamos que la `PEM` no es válida todavía porque el cuerpo Base64 está roto.
- `unable to load Private Key`: OpenSSL no pudo cargarla como clave RSA.
- `bad base64 decode`: El problema aparece antes de comprobar la estructura RSA: el texto entre `-----BEGIN RSA PRIVATE KEY-----` y `-----END RSA PRIVATE KEY-----`no formam un bloque Base64 correcto.
- **Hay que reconstruir la PEM ya que está incompleta.**

Reconstruimos la región de swap correspondiente al contenido de keys.txt

Como `crypy` escribía primero la pública y luego la privada, podemos reconstruirlo:

```
sed -n '15,49p' ~/analisis/swap_keys_region.txt >
~/analisis/keys_recuperado.txt
```

keys_recuperado.txt

Donde el `disassembly` ya confirmó esto:

- `rsa.newkeys(2048)`
- `public.save_pkcs1()`
- `private.save_pkcs1()`
- `open("keys.txt","w")`
- `write(public_pem)`
- `write("\n")`
- `write(private_pem)`

Y justo en swap aparece una secuencia:

- `BEGIN RSA PUBLIC KEY`
- `BEGIN RSA PRIVATE KEY`
- `cuerpo PEM`
- `END RSA PRIVATE KEY`

Eso encaja con un `keys.txt` borrado o desalojado a swap.

La clave private NO está completa

La private de swap_keys_region.txt está casi completa, pero le faltan caracteres al principio de la línea 26.

Flujo:

- **El keys_recuperado.txt contiene la `public` justo encima.**
- La `private` empieza con el mismo módulo `RSA`.
- La línea 26 de la `private` empieza en `Bnl2Rf2K...`, lo que indica que se ha perdido el prefijo de esa línea.
- NO hay que adivinar los caracteres que faltan: Aparecen de la `public` de arriba.
- Falta: `24LS0XY7IXmuh3e4yOm`.
- La línea completa es: `24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk`.

Explicación en detalle:

1. Vimos en el `keys.txt` recuperado que la clave private estaba justo debajo de la public: En swap_keys_region.txt, vimos exactamente ese patrón:

```
-----BEGIN RSA PUBLIC KEY-----
bloque Base64 de la pública
-----END RSA PUBLIC KEY-----
-----BEGIN RSA PRIVATE KEY-----
bloque Base64 de la privada
```

Eso nos indica que la public que está arriba y la private que estaba abajo pertenecían al mismo par de claves.

2. La línea rota de la private coincidía con el tramo final del módulo público: La private candidata empieza:

```
MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9
/1JZ1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU
Bnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
```

donde:

- La tercera línea empezaba directamente por: `Bnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk`.
- Eso no parece un inicio natural de línea Base64. Falta claramente un prefijo.

3. Miramos la clave public que estaba justo encima, y allí vemos este tramo continuo del mismo módulo:

```
...+qOijTZU
24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
```

Donde:

- Apostamos a que la línea correcta de la private debía ser: `24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk`.

LA PEM RECONSTRUIDA

candidata_swap_private_reparada.pem


```
-----BEGIN RSA PRIVATE KEY-----
MIIEqAIBAAKCAQEAqw9qsokSobI0TJxNdux1onuaFlUWBKl44g6wu2FP7LvRJZn9
/1JZ1rvM9KViZuACSUcaiT2FvhXuKXtU8xb8l+Mw+ZgZF4vyPljq5TdP+qOijTZU
24LS0XY7IXmuh3e4yOmBnl2Rf2KjJxyA5egOo0sEX9jn6BEMB3aATnzAsLqKnYGk
Y8kAjHOre6a1tRLNeLZZ5Tx5xY9JUOvRhxUdjAc5fYvDn6N2SaIcsO5H3P08XYst
3hseEFZc5E1DQ1ULzzGKR4MZiUBtzNfy/kGNRUXgWorI6VrJFMLga/2HufDtaqpB
gXMyNCEW3m/7bUZPCG8yeutBa91JDwtx6la5pwIDAQABAoIBACoQnHhS5NYGcH32
YcZA15yhMH8VZVi4JBXS7vrlLzZT6VaJy8AhjB4WCIyQruYFaAqFBa2CB4V69xoU
TguJN4xSPhTipij4LRXP39OGCpDj9HEtTyi7vwPrQxc0SQrfEf22Dj3XXJQquVxA
0iG3MwJMzEKVoFGc/2GyYT1i0+7b5G3YH65iEnqxDUtCmfawgeupKNA0QuTPZbLT
m5433oxHFYk1CM0RaM/Q5hv/Apk3h7BVnb36T3IHqsMfSvhQKf1FBrFSmMJqgasu
PaW93UZ1UsTLj9B/aAxz2BWl/69d3Q3NkX7rGU0Qj8JJM+2hjWFrBcXoiVV0lo6C
Xtqd4aECgYkArSVAU4F/SBUIZUPr1Ou+lC0cc/tAyv1z8+TW3/yjCOWW2O0f0SGc
M7/kOQ0S8UhlcoRWTBKSQlhRlx9t2qFUSbPU6aM5XVxeC4CkKcgRRxKPwHWsw4sk
KZaWRWoKoNc2btpr4sQwd28V4CWXTFynLpN3++rK3/UZlBHgH/E+hdD1Z2Y8CDBL
tQJ5APzqtldPqOqT+OtdYjC7FJCfb2V2Rb+1YZfPARoz2z4wZl7PBWeErNifrHni
5zMsYjF7aufURwJ9pHH+2/JnzjtF2Eh21lQA9khFyq+ydshw2O6L5RGbivshwqU1
SYnMyEWNz6JBaJFmNKDQNkcYuXn+2BxzwUnhawKBiEy6P0FCF5dnyqqlaLKYLYIY
Y416AAJ0QpgIBriHie/9XdWPbxQL4t6D4KzvgD9wJ5SR1HQtnqbL4Iwpn1LNPz7R
9D382QToU7yhKx5gzFdEgHN3MjINTN8pHjpwfXk9JaAGJtz8t8WVcoo73ko2nNdk
/jyilXidctJNAZbmHyDJspY5lczDKikCeEhU/BcWYjHMZfkyON3E80m1P0+eNq5w
OlwHSMJsqDwt7WkZ/eqwKBq/97zYJtUc0WxjZpysui49LqWnhNHb3EBZZlS7H+rg
Z48d3qzG9m+2zmPSwI17lZDjA97p0YFGftRdjU9TM4u51j1TruJAcPMx7i6c8F+n
xwKBiG9wwhmEu/9zGPVR5mgn1SY9FQHr6vMOn1ce0ydw3hQpeL3lfIXMSRVuv2uF
93Ggkjhm98HPJkSGf8RYlCNZspiIpIdSC2Nhq0X/T//KCq2z4NFcSxgW/sjTcLsk
mMCG3O2nvua9qPD4q2EF+IvhWzwoUO3ztI2rFEfMlpggNw93yzBvCezIvLg=
-----END RSA PRIVATE KEY-----
```

Validamos la clave:

```
openssl rsa -in ~/analisis/candidata_swap_private_reparada.pem -check -
noout
RSA key ok
```

La solución

Texto plano exacto recuperado es: `La palabra de paso es: gato-verde`.

- La `private key` de `keys.txt` estaba en swap, pero truncada en una línea, y se podía reparar usando el tramo correspondiente del módulo público que aparecía justo encima.
- `crypy` no es un `.py`, sino un `.pyc` renombrado.
- El flujo real es:
  - `rsa.newkeys(2048)`.
  - Guardar `public_pem + private_pem` en `keys.txt`.
  - Cifrar `./mysecrets.txt`.
  - Guardar `./mysecrets.txt.encrypted`.
  - Borrar `./mysecrets.txt`,
- La clave correcta no era la `PEM` repetida de RAM que habíamos probado antes.
- La buena estaba en `swap`, dentro del `keys.txt` borrado.
- El error concreto era una línea truncada en la `private`:
  - Se completa una línea truncada.
  - Faltaba el prefijo `24LS0XY7IXmuh3e4yOm`.

- Se pudo reconstruir a partir de la `public key` adyacente.

Conclusiones

Se realizó un análisis sobre una máquina Linux con el objetivo de recuperar la clave necesaria para descifrar `mysecrets.txt.encrypted` y obtener la flag.

En una primera fase se localizaron referencias en memoria a los artefactos del ejercicio, especialmente `crypy_decryptor.py`, `crypy` y `mysecrets.txt.encrypted`. Sin embargo, las primeras claves `PEM` encontradas en RAM no permitían el descifrado del fichero, por lo que fueron descartadas como claves ajenas al proceso relevante.

Posteriormente se localizó el fichero `/home/admin1/crypy`, que resultó ser un binario `Python 3.6 byte-compiled`. A partir de su análisis se comprobó que el programa:

- Generaba un par RSA de 2048 bits con `rsa.newkeys(2048)`.
- Guardaba `public_pem + private_pem` en `keys.txt`.
- Cifraba `./mysecrets.txt`.
- Escribía el resultado en `./mysecrets.txt.encrypted`.
- Eliminaba después el fichero plano original `./mysecrets.txt`,

El análisis del `bytecode` mostró que la clave correcta no debía buscarse en una PEM cualquiera de memoria, sino en los restos del fichero `keys.txt`. Como `keys.txt` ya no existía en disco, se examinó la `swap`. En `swap.strings` se localizaron fragmentos de una `RSA PUBLIC KEY` y `una RSA PRIVATE KEY` compatibles con el flujo del programa.

La clave privada recuperada desde `swap` estaba casi completa, pero contenía una línea truncada. Ese fragmento faltante pudo reconstruirse usando el tramo correspondiente del módulo público encontrado justo encima en la misma región de `swap`. Una vez reparada la `PEM`, se validó y se utilizó para descifrar `mysecrets.txt.encrypted` con la librería `RSA`.

Resultado final: `La palabra de paso es: gato-verde`.

La resolución del ejercicio no dependía de encontrar una PEM completa en RAM, sino de identificar el flujo real de `crypy`, comprobar que generaba y almacenaba las claves en `keys.txt`, y reconstruir desde swap una copia parcial de ese fichero.